claude-windows / c6028e72-ebfd-4218-89a8-c4d7dd23be96

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c6028e72-ebfd-4218-89a8-c4d7dd23be96\subagents\workflows\wf_49aab888-e10\agent-aa4195bf7c3ba5590.jsonl`
- SHA-256: `4949fd8fd89827f2cb477a507a29ecdf47b7b7b2abf82e4d616b7fcf006e267e`
- Source modified: `2026-06-08T13:04:39+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_49aab888-e10`
- session_id: `c6028e72-ebfd-4218-89a8-c4d7dd23be96`

Transcript

1. user (2026-06-08T13:03:33.578Z)

Read our roadmap/strategy/data docs and produce a tight grounding brief for an owned-model training study. Read what exists of: C:/Users/avidu/Projects/badminton-highlight-indexer/docs/PLATFORM_ARCHITECTURE.md, C:/Users/avidu/Projects/badminton-highlight-indexer/docs/COMMERCIALIZATION.md, C:/Users/avidu/Projects/badminton-highlight-indexer/docs/ROADMAP.md, C:/Users/avidu/Projects/badminton-highlight-indexer/docs/DEFERRED_HARDENING_PLAN.md, C:/Users/avidu/Projects/badminton-highlight-indexer/docs/archives/decisions/DECISIONS.md (esp. ADR-007), C:/Users/avidu/Projects/badminton-highlight-indexer/docs/archives/research/RALLY_DETECTION_RESEARCH.md, C:/Users/avidu/Projects/badminton-highlight-indexer/CLAUDE.md. Extract the OWNED-model intent, the hard guardrails, the data assets we can reuse, and how the roadmap frames the path from heuristics -> distilled -> Option-B end-to-end. Confirm/correct the context below against the docs.

OUR SITUATION (a local-first racket-sports highlight indexer + rally detector):
- GOAL: an OWNED, on-device-capable model for RALLY TEMPORAL SEGMENTATION (predict rally [start,end] intervals in a match video), evolving toward "Option B" = an OWNED END-TO-END MULTIMODAL model (frames + audio + pose -> rallies/highlights) fine-tuned on our own data. Privacy lever = local inference, user video never leaves the device.
- HARD GUARDRAILS (non-negotiable): (1) base/teacher models for TRAINING must be OPEN Apache-2.0 (or MIT/BSD) ONLY; video-capable bases like Qwen-VL / Gemma family are candidates BUT **Gemma 3 is REJECTED (viral/non-standard license)**. (2) NEVER use paid LLM outputs (Gemini/OpenAI/Anthropic) as TRAINING data ? they are inference/eval/teacher-for-eval only (terms + copyright). (3) Exclude minors from the training pool; biometrics/gait = future (GDPR Art. 9). (4) Generic data schema; coach->athlete is an optional overlay.
- DATA ASSETS WE WANT TO REUSE FOR TRAINING: human-labeled rally [start,end] CSVs per match (currently Adarsh-and-Avi 96 rallies, Kushagra 32, more coming; schema rally_number,start_time,end_time,ending_reason,sport) + per-rally ending_reason labels (winner/forced_error/unforced_error/service_fault/let) + per-video WASB shuttle-trajectory CSVs (Frame,Visibility,X,Y; from a frozen MIT-licensed shuttle detector) + (eval-only, NOT training) Gemini silver labels. Footage = single-camera amateur GoPro, 1080p/4K, 60fps.
- CURRENT PIPELINE: WASB (frozen vision shuttle detector) -> trajectory -> hand-tuned windowing + a net-crossing "gate" + a small distilled LogisticRegression on trajectory features. Honest results: threshold tuning ceilings ~F1 0.73 vs HUMAN labels on one match; a learned model UNDERPERFORMS baseline at n=2 videos. Bottleneck = more golden data + WASB recall, not the method.
- CONSTRAINTS: solo founder, BANGALORE-based (cost-sensitive; cloud GPU pricing + data-residency matter), local hardware = ONE RTX 2070 Super (8GB) + WSL2. User explicitly mentioned Vertex AI but wants MORE options + a detailed comparison.
- THE TASK CLASS is temporal action localization/segmentation (action = "rally"); labels are segment intervals.

2. assistant (2026-06-08T13:03:35.261Z)

I'll read the documentation to extract the owned-model intent, guardrails, data assets, and roadmap framing.

3. user (2026-06-08T13:03:35.698Z)

```
# Commercialization Tracker

**Status:** ? LIVING ? track this in parallel with product/quality work.
**Owner:** avidullu · **Last refreshed:** 2026-06-07

**Purpose.** Single source of truth for the **commercial** dimension of the project:
licensing/legal posture, codec & output economics, hosting unit-economics, the
golden-set vendoring spend, packaging/distribution, and the product-market-fit
risks that gate monetization. Updated as decisions land ? see the
[Changelog](#changelog) at the bottom.

**Merged from / supersedes:**
- `archives/MONETIZATION_AUDIT.md` (2026-05-30 deep-research licensing & economics
  audit) ? kept as the **research-of-record** (per-claim verification transcript,
  primary sources). This doc carries its *current* status forward.
- `archives/COMMERCIAL_READINESS_REVIEW.md` (code-health + PMF assessment) ?
  archived; its commercial findings + the quality-phase priorities live here now.

**Premise (unchanged, verified):** the product is a **closed-source hosted API** ?
users upload videos to a cloud NVIDIA GPU VM and get highlight reels back. We never
distribute code/binaries, so GPL/LGPL *distribution* copyleft is not triggered ? but
**AGPL §13 (network clause)**, **metered-API terms**, and **video-codec patent
royalties** all still apply.

---

## 1. Commercialization dashboard

Legend: ? done · ? in progress / watch · ? blocker / needs action · ?? needs legal counsel · ? future
Priority: **P0** (do first) · **P1** · **P2** (lowest). Rows are grouped **Pending ? In-flight ? Completed**, each sorted by priority. (C# IDs are stable identifiers ? they don't imply order.)

| # | Workstream | Pri | Status | Where it stands | Next action / owner decision |
|---|---|---|---|---|---|
| | **? PENDING (needs action / not started)** | | | | |
| C5 | **Output codec royalties** | **P0** | ? | Highlights are still encoded **H.264 + AAC** (`libx264` in `backend/pipeline/stitcher/compiler.py`) ? encode-side AVC patent exposure remains. | Switch output to **AV1 + Opus** (royalty-free) on an **Ada-class GPU (L4/L40)** for HW encode. See §4. Decision: provision Ada-class GPU? |
| C9 | **Packaging / distribution** | **P0** | ? | No `pyproject.toml`; the diagnostic `setup.py` shadows the build system. | Design in `DEFERRED_HARDENING_PLAN.md` (#7) ? **awaiting your approval** before code. |
| C12 | **SaaS foundations (async/auth/tenancy)** | **P1** | ? | Sync `/api/process` blocks on long videos; no auth/tenancy. | Multi-user/platform strategy in `PLATFORM_ARCHITECTURE.md` (pluggable storage/compute, tenancy, orchestration); async slice = `DEFERRED_HARDENING_PLAN.md` (#16). |
| | **? IN-FLIGHT (in progress / watch)** | | | | |
| C4 | **Runtime AI strategy (A/B/C + owned model)** | **P0** | ? | Live = **(C) Gemini 2.5 Flash, paid tier**. Long-term = **own a self-hosted generative video-QA model** (fine-tuned open VLM) as primary, **paid Gemini always-ready as fallback** via the provider registry. | Make "provider registry + paid fallback" a core principle; build the owned model **after** the scaffolding/flywheel are solid. See `PLATFORM_ARCHITECTURE.md` §5A. |
| C13 | **Beachhead validation** | **P0** | ? | Market research (`PMF_MARKET_RESEARCH.md`) recommends **India badminton academies** as the #1 beachhead. Unvalidated. | Cheapest tests: 10?15 academy/coach discovery interviews; rally-detection P/R benchmark on 100+ real amateur clips; fake-door pricing (B2C ~$99/yr, B2B academy ~$600/yr). See §6. |
| C8 | **Golden-set vendoring spend** | **P1** | ? | ADR-006: **Roora (Bengaluru)** selected for an initial paid pilot; licensed/owned video only (no user uploads to vendors). |
```

Confirm quality-per-dollar from the pilot; then GS-3 `HumanVendorProvider`. See `GOLDEN_SET_VENDORING.md`. |
C3	**Shuttle-tracking weights provenance**	**P1**	? ??	WASB-SBDT **code is MIT (clear)**; distributed **checkpoints are academic-data-trained** ? not covered by the MIT code grant. MonoTrack/YOLO-NAS banned (noncommercial).	Train-own weights (Phase-4 roadmap) **or** confirm per-checkpoint terms ?? before commercial deploy.
C14	**Owned-model base licensing + training-data policy**	**P1**	? ??	Owned video-QA model (PLATFORM §5A) must build on **Apache-2.0-clean, video-capable** bases ? **Qwen2.5-VL-7B/32B** (Apache; 3B/72B are *not*), **Qwen3-VL-4B/8B**, or **Gemma 4** ? to keep weights private, gate, and monetize. **Reject Gemma 3** (viral derivative clause ? permanent Google PUP), Llama Vision (naming + 700M-MAU), `gpt-oss` (text-only). **Hard rule: never train on paid-LLM (Gemini/OpenAI/Anthropic) outputs** (competing-model clause). The moat is the **consented dataset + eval harness**, not the model.	Lock to Apache-2.0; **open-model teachers only**; separate explicit **training opt-in**; **exclude minors** from the training pool; per-clip consent ledger. See PLATFORM §5A/§7.
C6	**Inbound HEVC decode**	**P2**	? ??	Decoding users' GoPro H.264/**HEVC** uploads is unavoidable; HEVC pools are fragmented + highest-uncertainty.	Low/uncertain residual; confirm decode-side exposure with counsel before launch.
C7	**Hosting unit-economics**	**P2**	?	Per-video cost is *cheap* for Gemini (~$0.05 input for a 10-min clip); risk is volume × latency, not unit price. Self-host/offline costs unmeasured.	**Benchmark $/video & latency on the target VM** for A/B/C before pricing. See §5.
C10	**Value layer (product surface)**	**P2**	?	Minimal exports shipped (EDL/JSON/CSV + `basic_stats`, PR #51).	Deepen: per-rally tags/notes that feed golden, per-player/match stats, clip gallery. See §6.
C11	**Input-quality enforcement**	**P2**	?	Per-video observability reports surface a customer verdict + footgun flags incl. audio-present (PR #50).	Turn more `VIDEO_RECORDING_GUIDELINES` dimensions into validators/report rules.
C2	**`supervision` ByteTrack pin**	**P2**	?	Pinned `<0.30.0` (deprecation ahead).	Migrate before lifting the pin (BACKLOG). Not a legal risk, a maintenance one.
	? COMPLETED				
C1	**AGPL / copyleft removal**	?	?	ADR-005 shipped: AGPL `ultralytics` YOLOv8 replaced with **torchvision Faster R-CNN (BSD) + `supervision` ByteTrack (MIT)**. No AGPL code/weights in the stack.	Done. Keep new deps permissive.

---

2. Licensing & dependency posture (refreshed risk table)

Verdicts updated against the current stack (`requirements.txt` + code). Original 2026-05-30 verdicts and the 25-claim verification transcript are in `archives/MONETIZATION_AUDIT.md`.

Dependency	License	SaaS obligation?	Current verdict	Notes
~~`ultralytics` + `yolov8n.pt`~~	AGPL-3.0	Yes (§13)	? **REMOVED**	Replaced per ADR-005 ? no longer in the stack.
torchvision detectors (Faster R-CNN/RetinaNet)	BSD-3	No	? LIVE	The chosen replacement; person = COCO class (see `yolo_hybrid.py` `_PERSON_CLASS_ID`).
`supervision` (ByteTrack)	MIT	No	? LIVE	Pinned `<0.30.0` ? migrate before bump (C2).
WASB-SBDT (shuttle)	MIT (code)	No	? code clear	Weights provenance open (C3).
MonoTrack / YOLO-NAS	noncommercial	?	? BANNED	Do not use.
Gemini API (paid tier)	service terms	Paid: no training/review	? LIVE WATCH	Must bill the paid tier to keep user video private (C4).
H.264 / HEVC / AAC codecs	patent pools	royalties on encode/decode	? ??	Separate from ffmpeg's license. Encode-side open (C5); decode-side residual (C6).
ffmpeg/ffprobe, google-genai, torch, opencv, numpy, fastapi/uvicorn/pydantic/jinja2/dotenv	LGPL/Apache/BSD/MIT	No	? CLEAR	Invoked as subprocess / imported; all permissive.
`obsreport` (reports)	permissive (private)	No	? soft dep	Pin the `git+ssh@vX` tag once the repo is published (BACKLOG).

Bottom line: the two hard blockers from the audit (Ultralytics; banned weights) are resolved/avoided. Remaining commercial-legal items are **codec royalties (C5/C6)** and **weights provenance (C3)**, plus the metered-API privacy posture (C4).

---

3. Runtime AI strategy ? A/B/C

The semantic rally-boundary step is the one external/metered dependency. Three paths (from the audit), with current standing:

- **(A) Eliminate ? offline learned segmenter.** ActionFormer (MIT) / our own learned classifier on `candidate_segments.stats`. **Best steady-state margin** (no per-call cost, no external dep) and matches the Phase-4 roadmap (substrate = WASB shuttle-motion, decided in ADR-004). **Most engineering.** ? *target*
- **(B) Self-host VLM.** Qwen2.5-VL-7B/32B (Apache-2.0) or InternVL2.5-8B (MIT). No per-call fees; pays in GPU VRAM (~16?24 GB for 7B). ?? Qwen 3B is noncommercial.
- **(C) Keep Gemini 2.5 Flash (paid).** Fastest, cheap per video, video-native; ongoing metered cost + external dependency. **? current live path.**

Hosted alternatives for (C) comparison: Anthropic Claude (no training on inputs/outputs; customer owns outputs; **no native video** ? weaker here); OpenAI (no training by default; ZDR available); Gemini remains strongest video-native.

---

4. Codec / output economics (the live gap ? C5)

The trap: ffmpeg being free (LGPL) does **not** grant the *patent* rights to the codecs it implements. Royalties attach to the act of encode/decode.

- **Encode side (we control it):** highlights currently emit **H.264/AAC**. AVC (Via LA) is first 100k units/yr royalty-free with a permanent "free to end-users over the internet" provision ? a small SaaS likely falls under thresholds, but ?? confirm.
- **Mitigation:** emit **AV1 (SVT-AV1, BSD + AOMedia royalty-free patent grant) + Opus (royalty-free)**. NVENC AV1 HW-encode needs **Ada-class GPUs (L4/L40)**; Ampere/Turing must use **SVT-AV1 (CPU)**.
- **Decode side (unavoidable, C6):** inbound GoPro **HEVC** decode ? fragmented pools, highest uncertainty; low residual.

Recommendation: provision an **Ada-class GPU (L4/L40)** and standardize highlight output on **AV1 + Opus** ? removes the encode-side royalty question almost entirely. *Implementation lives in `backend/pipeline/stitcher/compiler.py` (codec args).*

---

5. Hosting unit-economics (C7)

Reference workload: ~30-min 1080p match on a cloud NVIDIA GPU VM.

- **(C) Gemini 2.5 Flash (paid):** input $0.30/1M tokens, output $2.50/1M; video ~263 tok/s ? 10-min clip ? 158k input tokens ? **~$0.05 input**. Cheap per unit; the economic risk is **volume × latency**. Flash-Lite is cheaper; Pro ~10×.
- **(B) Self-host Qwen2.5-VL-7B:** $0 API; cost = GPU-hours / throughput; needs ?24 GB VRAM (L4/A10). $/video vs Gemini is **open ? measure**.
- **(A) Offline ActionFormer/learned:** cheapest at inference; cost = one-time training + light GPU. **Best steady-state margin.**

> The A/B/C $/video comparison depends on GPU instance + batch throughput ?
> **benchmark on the target VM before pricing the product.**

---

6. Product-market fit & the quality/data phase

Beachhead (from market research, 2026-06-07 ? see

[`PMF_MARKET_RESEARCH.md`](PMF_MARKET_RESEARCH.md))
The static-camera constraint is a **B2B filter**: the buyer must already record from a fixed angle, which points away from the median casual player and toward coaches/clubs.
Ranked beachheads:
1. **Badminton coaching academies & serious players, India** *(strongest)* ? 20M+ players, formalizing academy ecosystem (Khelo India), genuine fixed-angle recording, coaches who already monetize, badminton whitespace vs incumbents. Sell to the **academy**, not the individual; needs INR-tuned pricing.
2. **US/global pickleball players & clubs** ? fastest-growing sport (19.8M US, +45.8% YoY) but **SwingVision already owns single-camera tennis/pickleball AI** (USD 179.99/yr) ? head-to-head, not greenfield.
3. **Badminton academies/clubs in East/SE Asia + Denmark** ? highest density; China needs local hosting/partners; Denmark/EU small but high-WTP, easy to pilot.

Pricing anchors *(estimate)*: software-only BYO-camera **B2C USD 60?180/yr**, **B2B academy/club USD 300?1,500/yr** per location (undercuts hardware-bundled Veo/Pixellot/ Spiideo; comparable to SwingVision software-only).

PMF risks (from the readiness review)
- **Capture quality is the gate.** Accuracy is capture-conditional; broadcast/handheld footage is a failure mode. The wedge (static, tripod, full-court) is defensible but narrows the addressable market ? enforce/guide it (C11, `VIDEO_RECORDING_GUIDELINES`).
- **Value beyond raw segments is table stakes.** Time-saved editing alone is weak; per-rally tags, stats, and a correction loop are what make it sticky (C10).
- **Golden labels are the recurring blocker** across the offline segmenter, audio fusion, and measurable quality ? the highest-leverage investment (C8).

Refined priority order (quality/data phase ? "focus on quality once new videos land")
Carried from the readiness review; commercialization items above are tracked in parallel.

1. **Golden-set expansion + data-flywheel UX** (highest with new labeled videos): ingest the new batch, expand calibration/CV/regression baselines, make label contribution + safe retraining easy and non-contaminating (eval/training separation).
2. **Hybrid unification + observability hooks** (deferred item 7, now higher): extract shared logic from `tracknet_hybrid`/`wasb_hybrid` before heavy AI timing/metrics land.
3. **Detector/windowing quality iteration** with new data: re-tune velocity/merge/ min-duration + `rally_gate`; validate WASB on real video; trial audio fusion (ADR-007).
4. **Deeper value layer** (beyond minimal exports, C10): per-rally tagging/notes ? `candidate_segments`, per-player/match stats, clip gallery, human-correction loop.
5. **Input-quality enforcement & guidance polish** (C11): more guideline dimensions ? validators/report rules; "your video is good enough" UX.
6. **Multi-sport maturation + recording best practices.**
7. **SaaS / multi-user / async foundations** (C12) ? lower until monetization is firm.
8. **Polish & distribution** (C9): installer story, first-run path, codec/output, economics.

Recommendation: with the mid-week labeled-video batch, spend ~80% on #1 + #3 (flywheel + quality iteration on real data), then unification (#2) and deeper value (#4).

---

7. Open questions needing owner / counsel sign-off ??

1. **Ultralytics Enterprise price** ? only relevant if a permissive detector ever proves insufficient (fallback). Quote-only.
2. **Per-checkpoint / dataset licenses** for WASB / TrackNetV3 / any VLM ? train-own or

  confirm each (C3).
3. **Exact Via LA (AVC) / Access Advance (HEVC) rates & free thresholds** ? confirm live
   figures with counsel before relying on the "small SaaS under threshold" reading (C5/C6).
4. **Real per-video GPU cost & latency** of self-hosted VLM vs Gemini paid at expected
   throughput (C7).
5. **Provision Ada-class GPU (L4/L40)** for HW AV1 encode? (C5 ? gates the codec switch.)

Changelog

- **2026-06-07** ? Added the market-research-of-record (`PMF_MARKET_RESEARCH.md`):
  ranked beachhead (India badminton academies #1), pricing benchmarks, and a C13
  "beachhead validation" workstream. Wired the summary into §6.
- **2026-06-07** ? Created this living tracker. Merged the 2026-05-30 monetization
  audit + the commercial-readiness review. **Refreshed since the audit:** C1 AGPL
  removal now ? shipped (was the #1 blocker); added C5 as a ? (output is still H.264,
  the audit's AV1 recommendation is not yet implemented); C9 packaging tied to
  `DEFERRED_HARDENING_PLAN.md`; reflected PR #50/#51 (input-quality reports, value-layer
  exports). Archived `COMMERCIAL_READINESS_REVIEW.md`.

See also
- `PMF_MARKET_RESEARCH.md` ? market-study-of-record (sizing, segments, competitors, beachhead; cited).
- `archives/MONETIZATION_AUDIT.md` ? licensing/economics research-of-record (sources + verification transcript).
- `archives/COMMERCIAL_READINESS_REVIEW.md` ? superseded assessment (history).
- `archives/decisions/DECISIONS.md` ? ADR-005 (permissive stack), ADR-006 (vendor process).
- `ROADMAP.md` ? Phase-4 offline segmenter (the C4 "(A) eliminate" path).
- `DEFERRED_HARDENING_PLAN.md` ? packaging (C9), async (C12) designs (pending approval).
- `GOLDEN_SET_VENDORING.md` ? vendor process + Roora pilot (C8).
- `BACKLOG.md` ? `supervision` migration (C2), `obsreport` pin, codec follow-ups.

4. user (2026-06-08T13:03:36.003Z)

Roadmap ? from annotated data to a self-improving, offline rally segmenter

This document is the **narrative thread**: how the indexer evolves from an
AI-API-dependent prototype into a self-contained tool that learns rally detection
from its own commercially-licensed, annotated video corpus. It explains *how we
got here*, *why each step matters*, and *how it shapes the final tool*. Decisions
are recorded as ADRs in DECISIONS.md; the scoring mechanics live
in EVALUATION.md.

The thesis

The indexer detects badminton rallies in long recordings. Its strongest mode
(`yolo_hybrid`) is a two-stage pipeline: cheap **permissive person-detection
player-motion** filtering (Stage 1, local ? torchvision Faster R-CNN + ByteTrack
tracking) followed by **Gemini** semantic refinement (Stage 2, paid API). That
works, but it costs money, needs the network, and never improves on its own.

> *Naming note:* the `yolo_hybrid` registry key is retained for back-compat, but
> AGPL YOLOv8 was removed and replaced with a BSD/MIT-licensed torchvision +
> ByteTrack stack ? see **ADR-005**. "YOLO" in this doc means the local
> player-motion gating stage, not the Ultralytics library.

A sibling project, **sports-data-collector**, curates sports videos with
per-rally `[start, end)` annotations and a commercial-license flag. The thesis of

this roadmap: **use that annotated, commercially-usable data to (1) measure the pipeline objectively and (2) train an offline replacement for the paid Gemini stage** ? turning the tool into one that improves as more data is curated, runs air-gapped, and generalizes across sports.

Why it matters: it removes the per-run API cost and network dependency, gives us a defensible quality metric instead of eyeballing, and creates a flywheel ? more curated matches ? a better offline segmenter ? better highlights.

The four phases

Phase 1 ? Bridge the repos (DONE)
A sport-agnostic `curate export` in the collector emits the indexer's `start,end` ground-truth CSVs plus a `manifest.json` pairing each CSV with its video, gated on the `is_commercial` flag. ? *Why:* one clean producer/consumer contract that works for all five supported sports (badminton/tennis/cricket/pickleball/table-tennis) alike. See **ADR-002**.

Phase 2 ? Acquire full-resolution video (DONE)
The corpus was 256×144 (the original crawl used `worstvideo`). The `fetch` command re-acquires every video at **best available resolution (1080p here)**, **in place** (preserving annotations), via **authenticated Firefox Premium cookies** and yt-dlp's `default` client. ? *Why:* YOLO/shuttle detection needs real pixels; and the journey surfaced non-obvious traps (SABR streaming ? 144p; Chrome DPAPI; rate limiting) now encoded in the tooling. **22/22 fetchable matches are now 1080p.** See **ADR-003**.

Phase 3 ? Evaluate + tune (IN PROGRESS)
`run_phase3_eval.py` walks the manifest, processes each video, and scores detected rallies against the ground truth ? per video and corpus-aggregate (precision/recall/F1, mean IoU, boundary error), for both the `candidates` (raw detector) and `final` (refined) stages. ? *Why:* the candidates-vs-final split tells us *where* error lives ? a **detection** problem (candidates miss rallies) vs. a **segmentation** problem (candidates catch them, refinement is loose). That diagnosis decides where effort goes, and establishes the baseline any trained model must beat.

Phase 4 ? Learn an offline, API-free segmenter (NEXT)
Replace the paid Gemini refinement with a **lightweight classifier trained on the rally annotations**, consuming features the pipeline already computes and stores (`candidate_segments.stats`). No external API, no new labeling. ? *Why:* this is the payoff ? a free, offline, self-improving rally mode. The **substrate was chosen from data**: the ADR-004 update (2026-06-02) measured the YOLO/player-motion candidate stage at **recall ~0.05 @ IoU 0.5** on `shuttleset_8` (median best coverage of a rally only ~20%) ? too low a ceiling to build on. So the offline segmenter's substrate is **WASB shuttle-motion** (ADR-001), not player-motion. See **ADR-004**.

Complementary local cues (beyond the segmenter swap)

Once the substrate is WASB shuttle-motion, the next lever is **fusing additional fully-local cues** to raise rally recall/precision without reintroducing a paid API. The first one explored is **audio "thwack" hit-detection** (**ADR-007**, accepted as a direction 2026-06-07): a pure-numpy spectral-flux onset detector in `backend/pipeline/audio/`. The **E1 GO/NO-GO probe** has run on real GoPro audio ? verdict **AMBER / leaning-negative**: recall signal is real (hit-clusters recover WASB-missed rallies) but the classical-onset precision ceiling (~2.2×) is too weak to fuse cleanly, and a band-pass doesn't rescue it. The only path past that is a *learned timbre classifier* ? which, like Phase 4, is **gated on golden labels**. See `AUDIO_E1_FINDINGS.md`. Pose/court serve-gating is parked as the #2 cue.

The recurring blocker across Phases 3?4 and the audio work is the same: **more golden labels** is the single highest-leverage move (see the cross-cut note below).

How this shapes the final tool

- **Offline & free by default.** The production rally mode becomes a local model;

```
  Gemini is kept only as an upper-bound reference, not a dependency.
- **Self-improving.** Every newly curated, annotated match is more training signal;
  re-training is a local, deterministic step.
- **Measurable.** Every change is scored against the golden set, so "better" is a
  number, not a vibe.
- **Sport-generic.** The label ? feature ? classifier machinery and the export
  bridge already span sports; adding a sport is data, not new architecture.
- **Commercially clean.** Only `is_commercial` data flows through the bridge, so the
  trained model and any derived product rest on a licensable foundation.

## Status at a glance

| Phase | What | Status |
|------|------|--------|
| 1 | Export bridge (`curate export` ? manifest + CSVs) | ? Done |
| 2 | 1080p acquisition (`fetch`, in-place, cookies) | ? Done (22/22) |
| 3 | Eval + tuning loop (`run_phase3_eval.py`) | ? In progress |
| 4 | Offline learned segmenter (WASB shuttle-motion substrate) | ? Next (substrate decided: WASB, ADR-004) |
| + | Audio hit-detection cue (`backend/pipeline/audio/`, ADR-007) | ? E1 probe run ? AMBER, gated on golden labels |

> **Cross-cutting:** golden-set ground truth (which powers Phases 3?4 and on-demand regression testing)
> is produced via a vendor process + pluggable annotation provider ? see
> [GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md) and **ADR-006**. As of 2026-06-02 the annotation
> guideline is authored (collector `docs/annotation/`) and **Roora** (Bengaluru) is selected for an
> initial **3-sport pilot** (badminton, pickleball, table tennis). Input-capture quality requirements
> that bound achievable precision/recall are tracked in [VIDEO_RECORDING_GUIDELINES.md](VIDEO_RECORDING_GUIDELINES.md).
>
> **Observability:** every processed video now emits an always-on per-video diagnostics
> report (spec-driven warnings/errors with FAQ refs, soft-dependency on `obsreport`), so
> pipeline regressions and footguns surface without re-running eval ? see
> [CODE_MAP.md](CODE_MAP.md) §2 (reporting/observability).

```

5. user (2026-06-08T13:03:36.064Z)

```
# Platform Architecture ? Multi-User "Assistant Coach"

**Status:** ? Strategy / research ? for review. No code yet.
**Date:** 2026-06-07 · **Owner:** avidullu

**Purpose.** Map the path from today's **single-user, local** tool to a **multi-tenant
"assistant coach" platform**: users upload videos, get highlights **stored and
browsable by history**, and choose **where their data lives** (Google Drive / cloud
bucket / local) and **where the compute runs** (our GPU / their VM). The organizing
principle is to **decouple the layers** ? identity, ingestion, storage, orchestration,
execution/compute, the detection+highlight pipeline, and data-serving ? behind small
**pluggable interfaces**, reusing the registry pattern the codebase already uses for
segmenters/providers/validators/sports/annotations.

**Product framing (the north star *and* the v1 line).** This is an **assistant coach**,
not just a clipper ? but the strategy is **phased and unambiguous about scope**:
- **v1 (concrete, near-term):** **rally detection ? highlight-reel compilation ? per-user
  history ? a coach/user correction loop that feeds the golden set** ("assistant-coach
  *lite*"). Essentially today's pipeline made multi-user ? this is the real v1 deliverable.
- **Mid-term:** **rich rally/shot semantic metadata** (shot type, rally outcome) and the
  **owned video-QA model** (§5A; §6 P4?P5).
```

- **North-star vision (strictly future ? *not* v1):** **gait / biomechanics, per-athlete
  identifying templates, and deep coaching feedback under a supervising coach** (§6 P6).
  Inspiring, but explicitly deferred ? and the part that raises the **biometric** privacy
  bar (§7).

The buyer is the **academy/coach** (beachhead in
[`PMF_MARKET_RESEARCH.md`](PMF_MARKET_RESEARCH.md)); **power users self-serve**. Two
consequences ripple through the design: (1) the data model keeps an **optional coach ?
athlete overlay over a generic core** (§4.1) ? not baked in; (2) gait/biometrics is
special-category data, gated and deferred (§7).

> **Competitive note (validated by research):** none of Veo / Hudl / SwingVision /
> Spiideo offer true BYO-storage or BYO-compute ? they're vendor-cloud SaaS
> (SwingVision uniquely runs *edge* compute on the user's iPhone). So **"your video,
> your storage, your compute"** is a genuine differentiator for privacy-sensitive
> academies ? and SwingVision proves user-side compute is commercially viable. All four
> share the same **teams ? athletes ? clips** data model, which we adopt (§4).

> Scope: this is a **direction-setting doc**. It does not start implementation; the
> first infra slice (an async job model) is the on-hold #16 in
> [`DEFERRED_HARDENING_PLAN.md`](DEFERRED_HARDENING_PLAN.md). Commercial constraints come
> from [`COMMERCIALIZATION.md`](COMMERCIALIZATION.md) (C4 paid-LLM no-train; C12 SaaS
> foundations) and `archives/MONETIZATION_AUDIT.md`.

1. Current state & gaps (what we're evolving from)

Concern	Today	Gap for multi-user
Identity / tenancy	none	no users, orgs, auth, or isolation
Storage	local `filepath` string; output to `output_dir`	no abstraction; can't BYO Drive/bucket; raw video must be on the box
Data model	`videos` keyed by file **MD5**, `play_segments`, `events`, `candidate_segments` (SQLite)	no owner/athlete link; **highlights aren't a first-class persisted entity** (compile writes a file and returns a path); no per-user history
Execution	pipeline runs **in-process, synchronously** in `/api/process`	blocks; can't dispatch to a user's VM/GPU; no job status
Pipeline	**already pluggable** (segmenter/provider/validator/sport/annotation registries)	reuse as-is ? this layer is the model for the rest
API	`/api/process`, `/api/query`, `/api/compile`, `/api/config`; no auth	needs auth, per-tenant scoping, async job + history endpoints

Grounding: `backend/infrastructure/database.py` (schema + `PRAGMA user_version`
migration ladder, v1), `backend/main.py` (endpoints, sync flow), the registry pattern
in `backend/pipeline/segmenters/base.py::SegmenterRegistry` and
`backend/providers/base.py`.

2. Target layered architecture

Decouple into layers with a clean **control-plane vs data-plane** split ? the
orchestrator coordinates *metadata and jobs*; the heavy *bytes and GPU work* can live
elsewhere (even in the user's own cloud).

```
                ?????????????????????????????????????????????????
   CONTROL      ?  Presentation / API  (auth, per-tenant)       ?
   PLANE        ?  Identity & Tenancy  (org?coach?athlete?user)  ?
                ?  Orchestration / Jobs (submit?status?result)   ?
                ???????????????????????????????????????????????????
                                ? dispatch       ? metadata
                ?????????????????????????  ???????????????????????
   DATA         ?  Execution / Compute  ?  ?  Data / Serving     ?
   PLANE        ?  (local | BYO-VM |    ?  ?  (per-tenant DB:    ?
                ?   hosted GPU)         ?  ?   videos, segments, ?
```

```
                    ?   runs the Pipeline:  ?  ?   highlights, jobs, ?
                    ?   detect ? highlight  ?  ?   history)          ?
                    ??????????????????????????  ???????????????????????
                                ? read input / write artifacts
                    ?????????????????????????????????????????????????
                    ?  Storage  (local upload | Google Drive | S3 |  ?
                    ?  GCS | Azure)  ? raw video + highlight outputs ?
                    ??????????????????????????????????????????????????
```

The two **new pluggable seams** are **Storage** and **Compute/Execution**; everything else either exists (pipeline) or is net-new infra (identity, orchestration, serving). A **model/training layer** ? the owned generative video-QA model (§5A) ? sits atop these, fed by Storage (training corpus), Data (labels), and Compute (training + inference), and is delivered through the existing **AI-provider registry with a paid fallback**.

3. Pluggable abstractions (extend the existing registry pattern)

The codebase already proves the model: an `ABC` + a `Registry` singleton with `register/get/list_available` (e.g. `SegmenterRegistry`). Add two more registries with the same shape so they're plug-and-play and testable in isolation.

3.1 `StorageBackend` registry
Replace the bare `video_path: str` that flows through the pipeline with a **`StorageRef`** (a backend id + a key/URI), resolved by a backend.

```python
class StorageBackend(ABC):
    name: str
    def open_read(self, ref: StorageRef) -> BinaryIO: ...      # stream large video
    def download_to_local(self, ref: StorageRef, dst: str): ...  # for ffmpeg/cv2/WSL
    def upload(self, local_path: str, ref: StorageRef): ...     # persist highlight/artifacts
    def signed_url(self, ref: StorageRef, ttl: int) -> str: ... # hand to LLM / browser
    def stat(self, ref: StorageRef) -> StorageStat: ...
    def list(self, prefix: StorageRef) -> list[StorageRef]: ...
```

- **Recommended implementation:** build the backend over **`fsspec`** (one matrix ? local / S3 (`s3fs`) / GCS (`gcsfs`) / Azure (`adlfs`) + Google Drive via `gdrivefs`, Dropbox via `dropboxdrivefs`), reaching for **`smart_open`** as the robust large-MP4 byte-streaming primitive. Storage becomes one more registry. (Treat Drive/Dropbox as add-ons ? less battle-tested than `s3fs`/`gcsfs`; cover with integration tests.)
- **Backends:** `local` (upload dir), `gdrive`, `s3`, `gcs`, `azure`. **BYO-storage**: the tenant registers their own bucket/Drive; raw video **never leaves their account**.
- **Auth ? never persist users' long-lived keys.** For buckets, have the user create an IAM role you **`AssumeRole`** into, scoped to one bucket/prefix, and mint **presigned URLs** so raw video moves **browser ? the user's bucket directly, never through our servers**. For Drive, OAuth with the narrow **`drive.file`** scope; for academies, a **service account with domain-wide delegation**. Store any per-tenant creds encrypted.
- **Why it matters for the pipeline:** ffmpeg/OpenCV/WSL need a **local file**, while the LLM provider can take a **signed URL**. The backend exposes both `download_to_local` (for compute) and `signed_url` (for the provider/browser), so the pipeline stops caring *where* the bytes live.
- **Migration path:** introduce `StorageRef` alongside the current local path; a `local` backend wraps today's behavior so nothing breaks; adapters land incrementally.

3.2 `ComputeBackend` / `ExecutionBackend` registry
Where the pipeline actually runs for a given job.

- **`local`** ? in-process / local subprocess (today's behavior; dev + self-host).
- **`hosted`** ? our GPU VM pool (the SaaS default).
- **`byo_worker`** ? the tenant runs a **pull-based worker** (a small agent) on their

own VM/GPU. **Critical security property:** the worker *pulls* jobs scoped to its tenant from the control plane, pulls input from the tenant's **own** storage, runs the pipeline, and writes artifacts + results back ? so **the orchestrator never holds the user's cloud credentials** and needs **no inbound access** to their network. This mirrors the **self-hosted CI runner** pattern (GitHub Actions runners use a 50s outbound-only HTTPS long-poll). This pull-based runner is the **cheapest, most credential-safe v1** for BYO-compute.
- For users who want the platform to **provision GPUs in their *own* cloud account on demand** (an academy with an AWS account but no standing GPU box), add **SkyPilot** (BYOC: everything launches in the user's accounts/VPCs; creds stay with them). **Modal** is the managed fallback for users wanting zero infra (accepting that raw video then flows through Modal).
- **?? Operational / support burden (a real adoption risk).** For the primary beachhead ? Indian badminton **academies/coaches** ? asking them to *run and maintain a worker on their own VM/GPU* is a meaningful ongoing load (updates, monitoring, firewall/NAT, the "it stopped after our IT change" ticket). So: **default academy users to `hosted`**, and treat `byo_worker`/SkyPilot as a **power-user / enterprise / privacy-sensitive tier**, not the academy default. If/when BYO ships, plan for **auto-updating, self-healing agents + health heartbeats + clear worker status in the UI**, and budget the support cost.
- **Middle tier to consider ? "zero-infra BYO":** a managed-but-isolated path where the **storage** stays in the user's bucket (signed URLs) while compute runs on **our** GPUs in a **strictly isolated, per-tenant, ephemeral sandbox** (nothing persists our side). This gives the BYO *privacy* benefit (raw video never persists on our infra) **without** the user running a worker ? a pragmatic default between full-hosted and full-BYO.

The pipeline code is unchanged; only *where it's invoked* and *how it gets bytes* vary.

3.3 Reuse, don't reinvent
The existing `segmenter` / `provider` / `validator` / `sport` / `annotation` registries are the proven precedent and stay as the **pipeline** layer. The `annotation` registry + `candidate_segments` table are also the natural seam for the **coach-correction / golden-set** loop in the assistant-coach phase (§6, P4).

4. Multi-tenant data model & serving layer

4.1 Entities ? generic core, with the coach model as an optional overlay
The foundational schema is deliberately **domain-neutral** so we don't overfit to the coach mindset ? the coach/athlete framing is a **market-fit hypothesis that will iterate** (see C13 / PMF), not a load-bearing assumption. **Generic core:**
```
User ??< VideoCollection ??< Video ??< Highlight ??< HighlightMetadata
                                 ???< PlaySegment ??< Event
Video / Highlight ??< Job (processing record: status, compute backend, timings)
```
- **Generic & re-targetable:** a `VideoCollection` groups a user's videos (a season, an event, a "folder") with no assumption of a coach or academy ? a power user, a coach, or a parent are all just `User`s with collections.
- **Highlights are first-class** (today compile only writes a file): a `highlights` table (segment ids + output `StorageRef` + params + created_at) powers "see past videos and highlights by history," and **`HighlightMetadata`** carries the **rich semantic tags** (shot type, rally outcome, etc. ? see §6 metadata stage) plus titles / ratings / notes.
- **`video_id` stays the content MD5** (dedup within a tenant), scoped by `tenant_id`.
- **Coach overlay (optional, additive):** for the academy/coach GTM, layer `Org/Academy ? Coach ? Athlete` *on top* (e.g. a `Membership`/role table + an `athlete_id`

  tag on collections/videos) **without** baking it into the core tables ? so the schema stays usable for solo power users and the coach model can evolve, or be dropped, as PMF dictates.
- **History/serving API:** list collections/videos/highlights per user (and, with the overlay, by athlete/coach/date); **query highlights by `HighlightMetadata`** (e.g. shot type / outcome); re-stitch from stored segments without reprocessing.

4.2 Tenancy isolation ? options & recommendation
Model	Isolation	Ops cost	Best for
Shared DB + `tenant_id` (row-level, ideally Postgres **RLS**)	logical	low	**Hosted SaaS, many small tenants** ? *recommended default*
Schema-per-tenant	medium	medium	tens?hundreds of mid tenants
DB-per-tenant (incl. **SQLite-per-tenant**)	strong/physical	higher at scale	**self-hosted single-academy**, or high-isolation enterprise

Recommendation: for the hosted product, **migrate SQLite ? Postgres with shared-DB + `tenant_id` + Row-Level Security**; keep **SQLite-per-tenant** as the **self-hosted / single-academy** deployment (it reuses today's code almost verbatim). Either way, **abstract DB access behind a repository layer now** so the engine is swappable, and graduate the `PRAGMA user_version` migration discipline to **Alembic** under Postgres.

5. Orchestration / async jobs

This is the serving layer that makes multi-user real. It is the **multi-user superset of [`DEFERRED_HARDENING_PLAN.md`](DEFERRED_HARDENING_PLAN.md) #16** (async job model) ? build that first, single-tenant, then generalize.

- **Job record** (`jobs` table): `id, tenant_id, video_id, type (index|compile|analyze), status (queued|running|done|failed), progress, compute_backend, result_ref, error, timestamps`. `/api/process` returns **202 + job_id**; `/api/jobs/{id}` polls.
- **Idempotency** keyed on `(tenant_id, video_id, pipeline_version)` ? reuse the content-hash discipline already in `compute_video_id`.
- **Dispatch:** the orchestrator enqueues; a worker (hosted or `byo_worker`) claims jobs for its tenant, resolves input via the tenant's `StorageBackend`, runs the pipeline, writes artifacts + DB rows. **Observability reports** (already per-video) become the job artifact surfaced in history.
- **Queue/worker backend ? a decision to prototype in #16, *not* a default here.** The *requirements* are firm ? **idempotency**, **resumability** (a 40-min GPU job mustn't restart on a worker blip), **progress states** ? which rule out fire-and-forget Celery/RQ for the long jobs. The *choice* should be made by prototyping in the [`DEFERRED_HARDENING_PLAN.md`](DEFERRED_HARDENING_PLAN.md) #16 slice, weighing:
  - **Start simplest:** **Arq** (async-native, natural FastAPI fit) or even a **plain Postgres-backed queue + worker** ? likely sufficient for the *first* single-tenant async slice, with the lowest operational complexity and the easiest debugging.
  - **Durable-execution upgrade if/when needed:** **Hatchet** (Postgres-backed, per-step retries/timeouts/concurrency; pairs neatly with the pull-based worker) ? attractive because it adds **no new datastore** once we're on Postgres, but it *is* a new dependency + operational/debugging surface; adopt only when long-job resumability/ observability demands it.
  - **Heaviest:** **Temporal** ? most mature durable workflows, operationally heavy; reserve for genuinely complex multi-stage pipelines at scale.

  Record the decision criteria in #16: operational complexity, maturity, **debugging experience for long GPU jobs**, and whether durability is worth a new dependency. **Bias: start with the simplest thing that gives retries + status; graduate only on evidence.**

5A. Owned generative model (the video-QA assistant) & the data flywheel

The north-star capability: an **owned, self-hostable generative video model** ?

fine-tuned on sports video (including **consented, user-contributed** footage) ? that **answers questions about rallies, shots, and technique**. This is the conversational brain of the "assistant coach." Crucially, it does **not** replace the known stack; it is introduced **behind the same abstraction, with the paid API as an always-ready fallback**.

> **The moat is *not* the model.** Anyone can download Qwen-VL/Gemma. The defensible
> assets are **(a) the consented, badminton-specific video?QA dataset** and **(b) the
> eval harness**. Build and message *those* ? not "we own an LLM."

> **? Committed guardrail (legal, non-negotiable).** We respect the licensing findings in
> full. **Paid frontier models (Gemini / OpenAI / Anthropic) are used for *inference /
> serving / fallback* only ? never as a *training* data source.** Harvesting their outputs
> to train our owned model would breach their "no competing model" terms (a terms/copyright
> violation), so it is prohibited in both engineering and legal process. **Training teachers
> are open, Apache-2.0 models only** (Qwen-VL / InternVL / Gemma 4). Detail in the
> distillation constraint below.

Core principle: AI is a pluggable provider with a paid fallback
The codebase already abstracts the cloud AI behind `provider_registry` / `AIVideoProvider` (`backend/providers/`). Make that a **first-class platform principle**:
- Every generative capability routes through the **provider registry**.
- The **owned model** plugs in as a new provider (`owned_vlm`); **Gemini (paid, via Vertex AI)** stays registered as the **fallback** (failover on error/low-confidence, or as the quality ceiling/teacher reference).
- This lets us **ship today on the known paid stack** and **swap in the owned model when its quality clears a measured bar** ? with zero architectural churn and the paid path always one config flip away. (Mirrors the COMMERCIALIZATION C4 A/B/C strategy: the owned model is the long-run "(A) eliminate the per-call dependency"; paid is "(C)".)

The model / training layer (new sub-layer, fed by the scaffolding)
```
Data curation        Training             Eval                 Serving
(consented user vid  (fine-tune an open,   (extend the existing (self-host inference
 + golden set +      commercially-usable   eval harness to QA   as a provider /
 rally/segment/event base VLM via          quality + the        ComputeBackend;
 annotations ?       LoRA/QLoRA)           regression gate)     paid fallback)
 video-QA pairs)
```
This layer **reuses every other layer**: **Storage** holds the training corpus; **Data** supplies labels (the `annotation` registry + `candidate_segments` + golden set are already the labeling seam); **Compute/Orchestration** runs training and inference jobs (even **BYO-compute** can serve inference on the tenant's GPU); the **eval harness** already in `backend/eval/` extends to grade QA quality. **In other words, the rock-solid scaffolding is exactly what makes the model flywheel possible** ? which is why scaffolding comes first (see §6).

Base model & training (Apache-2.0 only ? licensing is decisive)
Recommended near-term base: `Qwen2.5-VL-7B-Instruct` ? Apache-2.0, native long-video (dynamic-FPS sampling, temporal mRoPE), mature LoRA tooling. Upgrade path **Qwen3-VL-4B/8B (Apache-2.0)**; **Gemma 4 multimodal (Apache-2.0, 2026)** is a clean Google-stack alternative to evaluate.
- ?? **Lock to Apache-2.0-clean bases** so you can fine-tune, **keep weights private, gate behind subscriptions, and never owe open-sourcing or branding**. (Note Qwen2.5-VL is Apache **only at 7B/32B** ? 3B/72B use the custom Qwen license.)
- ? **Reject for the flagship:** **Gemma 3** ? custom terms with a **viral derivative clause** (anything fine-tuned on it, *or even trained on its synthetic outputs*, stays permanently bound to Google's *updatable* Prohibited-Use Policy); **Gemma 4's switch to Apache-2.0 removes this**. **Llama 3.2 Vision** ? custom license forces a "Llama-?"

model name + "Built with Llama" badge and a 700M-MAU ceiling. **`gpt-oss`** ? Apache-2.0 but **text-only**, so usable only as an optional text-reasoning layer, never the video encoder.
- **Training method:** **QLoRA** (? r=64, ?=128) on the 7B ? fits one prosumer GPU and avoids catastrophic forgetting of general video skills. Build the dataset by converting the existing rally/shot/segment annotations + golden set into templated **clip?QA pairs**, expand synthetically with **open-model teachers only** (see constraint below), and hold out a **golden QA eval** that gates every fine-tune. A few hundred gold pairs ? **5k?20k** curated is a realistic band for this narrow domain.
- **Serving economics:** a 7B VLM ? 14 GB VRAM (FP16) / ~3.5 GB (INT4); a ?16 GB GPU is a comfortable serve target (vLLM for throughput). Self-hosting wins **only** once you have sustained video-QA volume **+** a defensible consented dataset **+** real per-call API cost pain ? below that, the paid API is cheaper than a standing GPU and the second stack is overhead. Hence: **paid-primary now ? owned-primary / paid-fallback later.**

Base-model licensing + training-data policy are tracked as **COMMERCIALIZATION C14**.

Training backend ? managed service vs self-managed (the fine-tune "forge")
Where you run the LoRA/RL fine-tune. Weigh on **video support, weight export/ownership, smallest servable model, RL support, no-train DPA, and infra burden**.

Decisive finding (researched 2026-06): managed fine-tuning services are **image-based** (JSONL + base64 images) ? for video you pre-sample frames and feed them as image sequences. **Only the self-managed `ms-swift` explicitly advertises *native* video (+audio) training.** So for true video-QA, a self-managed framework is the strongest technical fit ? and it also lets you fine-tune the **small 7B** (cheap to serve) and keep full ownership. Managed services trade some of that for zero infra.

Option	Type	Video / VLM support	Smallest VL	RL	Export weights	Best for
ms-swift (ModelScope)	Self-managed (rented/own GPU)	**Native video+audio**; Qwen3-VL/Omni, 300+ MLLMs	any incl **7B**	? DPO/GRPO	? full ownership	**? video-QA; slots into BYO-compute (§3.2)**
LLaMA-Factory	Self-managed	Qwen3-VL + 100+ VLMs; video datasets	any incl 7B	?	?	popular, well-documented
Unsloth	Self-managed (1-GPU optimized)	Qwen3-VL LoRA (vision+lang; frames)	any incl 7B	? GRPO	?	cheapest first runs
Tinker (Thinking Machines)	Managed API (low-level)	Qwen3-VL **30B-A3B+ only**; image input (video **unverified**)	30B-A3B MoE	? PPO/GRPO, custom loss	? adapters + ckpt; no-train ToS	zero-infra RL/research at MoE scale
Together AI	Managed (tune + serve)	Qwen3-VL / Gemma-3 / Llama-4; image data	verify	partial	? adapters downloadable	managed Qwen3-VL + serving
Fireworks AI	Managed (tune + serve)	Full Qwen2.5-VL (3B?72B SFT; LoRA@32B + LoRA-upload)	**7B**	partial	?? verify export (lock-in risk)	managed *small*-VL + serving
Predibase (Rubrik)	Managed **+ VPC self-host**	VLM instruct-tune; **LoRAX** multi-adapter	verify	? (VPC)	? **RFT/GRPO** flagship	RL "creativity" + multi-adapter + privacy
~~OpenAI GPT-4o / Gemini tuning~~	Closed	?	?	?	? no export	**non-fit ? violates "own the model"**

(Enterprise clouds ? Databricks Mosaic / SageMaker / Vertex on open models ? give full control but are only worth it if you're already on that stack.)

Recommendation:
- **Early iteration / true video-QA ?** self-managed **`ms-swift`** (or LLaMA-Factory / Unsloth) on a **rented GPU** (RunPod / Lambda / Modal / SkyPilot-to-your-cloud):

native
  video, any size incl. **7B** (cheap to serve), full ownership, and it drops into the
  ComputeBackend / BYO-compute seam (§3.2). **Likely a better fit than Tinker** for
  the video requirement *and* serving economics.
- **Managed, no GPU ops ?** **Together AI** (Qwen3-VL + downloadable adapters + serving)
  or **Fireworks** (full Qwen2.5-VL incl. 7B + serving) ? confirm frame/video support and
  adapter export.
- **If RL "coaching creativity" + per-sport multi-adapter is central ?** **Predibase**
  (RFT + LoRAX, VPC self-host for the biometric/privacy posture) or **Tinker** (lowest-
  level RL primitives).
- **Tinker** remains a strong zero-infra RL-research option with clean weight export ? just
  not the cheapest path to a *small video* model (its VL floor is 30B-A3B).
- **Portability is the hedge:** you own the **data + recipe + exported weights**, so the
  backend is swappable ? same ethos as the provider/registry pattern. Don't get locked in.

Verify before committing (any backend): (1) **native video / frame-sequence**
ingestion + max frames per example; (2) **weight export** + a **commercial DPA** covering
consented, minors-excluded video (biometric, §7); (3) the **inference plan** for the size
you train (a self-managed **7B** is far cheaper to serve than a 30B-A3B MoE).

Training harness ? make it agent-runnable (nanochat-style operability)
The goal: **an agent (or CI) can run a fine-tune on a fresh batch of golden-labeled video
and get a trustworthy ship / no-ship verdict, unattended.** Karpathy's **nanochat** is the
reference for that *operability* ? one MIT repo, one `speedrun.sh` that runs
tokenizer?train?eval?serve, a single complexity dial, and an automatic **report-card**
score. It is **not** a usable engine here (text-only, from-scratch GPT-2-class), but its
five properties are exactly what make training agent-runnable, and we copy them:
(1) one command, end-to-end; **(2)** one complexity dial (sane auto-derived defaults);
(3) an automatic **report card** ? a deterministic pass/fail vs a target (the crux);
(4) reproducible + deterministic + cheap; **(5)** minimal/readable.

Engine (don't reinvent): **`ms-swift`** already delivers this for VLMs ? CLI-driven,
native video, Qwen3-VL support, an eval backend (EvalScope), and reproducibility via
`args.json` + `full_determinism`. (LLaMA-Factory is the close second; HF **`nanoVLM`** ?
~750 lines ? is the minimal reference to *learn* VLM training, not the production path.)

The harness = thin glue over components we already have ? one declarative,
agent-invokable job (call it `train_speedrun`):
1. **Build dataset** from golden labels ? clip?QA pairs (reuse the `annotation` registry +
   `candidate_segments` + golden set ? the labeling seam in `backend/`).
2. **Fine-tune** via ms-swift on a **ComputeBackend** GPU (§3.2), run as an orchestration
   job (§5 / DEFERRED #16): one config, idempotent, logged.
3. **Auto-eval + report card** ? extend the existing `backend/eval/` regression harness to
   QA quality; emit a `report.md` pass/fail vs the **held-out golden QA eval**.
4. **Register** the result as the `owned_vlm` provider (Gemini paid fallback stays) **only
   if it beats the bar**.

So: **golden labels ? ms-swift train (on a compute backend) ? eval gate ? register
provider**, behind one command an agent or CI runs. nanochat is the inspiration for the
wrapper; ms-swift is the engine; the **golden QA eval is the report card**.

> **Prerequisite (the crux):** the harness is only as trustworthy as the **golden QA eval

> set**. nanochat's speedrun is meaningful *because* its eval (CORE score) is automatic and
> trustworthy; ours is the golden QA eval ? **build that first**, or an agent-run loop
> optimizes blind. (Same "moat = dataset + eval", scaffolding-first principle.)

?? Teacher / distillation constraint (the single biggest legal landmine)
You **cannot** train the owned model on outputs from the paid frontier APIs ? a general video-QA assistant is squarely a *"competing model"*: **Gemini** ("may not use the Services to develop models that compete"), **OpenAI** ("may not use Output to develop models that compete"), **Anthropic** (bars using Outputs to train competing models). Two safe lanes: **(1)** generate synthetic QA with **open-model teachers only** (Qwen-VL / InternVL / Gemma 4 ? Apache-clean); **(2)** keep the paid API strictly as a **runtime fallback** ? serving answers is fine, *harvesting its outputs to train the model that replaces it is not*. Write a hard **"no training on competitor-API outputs"** rule into eng + legal process. (This makes the ROADMAP's "Gemini as offline teacher only" precise: the teacher is *open models*, not the paid APIs.)

Data flywheel, personalization & consent tiers
Two complementary loops, each with its **own** opt-in (training consent is separate from processing consent ? §7):

- **Global model (shared ? raises the floor).** Train the base owned model on an **explicitly-contributed, licensed** corpus ? *not* on whatever sits in a tenant's bucket (preserves "your video, your storage"). Flywheel: *more consented, labeled matches ? a better shared model ? better highlights/answers ? more reason to contribute.* Higher consent bar (pooled data, biometric/minor care ? §7); **minors excluded by default**.

- **Per-user adapters (personalized ? raises the ceiling) ? owner hunch, under exploration.**
  A small **per-user (or per-tenant) LoRA adapter, fine-tuned *only on that user's own consented videos*** on top of the shared base, served via **multi-LoRA** (LoRAX / Together / Predibase). Why it's compelling:
  - **Precision:** adapts to *that user's* court, camera angle, lighting, and players' styles ? materially better detection/highlights for them.
  - **Cleanest consent story:** *"improve **your** detection using **your** data"* is far lower-friction and lower-risk than contributing to a global model ? the data improves **only the user's own adapter**, with **no cross-user mixing**, sidestepping much of the pooled-corpus biometric/minor entanglement. Reinforces the differentiator: *your video, your storage, your compute ? and now **your model**.* With BYO-compute the adapter can be trained/served on the user's own GPU and **never leave their boundary**; **deletion = drop the adapter** (clean erasure).
  - **Open questions (still a hunch):** cold-start (need enough of the user's labeled data before a personal adapter beats the base ? fall back to the shared model meanwhile); a **per-user held-out eval** to prove the adapter helps (don't ship a regression); base + optional global-adapter + per-user-adapter stacking; per-user training cost at scale. Worth a spike once the harness + golden eval exist.

Two consent tiers, then: **(a)** "improve **my own** detection with **my** data" (per-user, low friction) and **(b)** "contribute to the **shared** model" (global, higher bar). **Default for both is off.**

6. Phased roadmap

Phase	Deliverable	Build / reuse
P0	**Async job model, single-tenant** (DEFERRED #16)	`jobs` table via `user_version` ladder; worker; 202+status endpoints
P1	**Identity + multi-tenant data model + upload + highlight persistence + history**	auth (build vs Auth0/Clerk/Supabase ? decide); Postgres + `tenant_id`/RLS; `highlights` table; repository layer; history API + UI
P2	**`StorageBackend` registry + BYO-storage**	new ABC+registry; `local`?`gdrive`/`s3`/`gcs`; replace `video_path` with `StorageRef`; encrypted per-tenant creds
P3	**`ComputeBackend` registry + BYO-compute**	`local`/`hosted`/`byo_worker`; pull-based worker agent; control/data-plane split
P4	**Rich rally/shot semantic metadata & tagging** *(the next value stage after solid rally detection + highlights)*	shot type (smash / backhand drive / net), rally **outcome** (winner / forced / **unforced error**), context (e.g. **open court but error**) ? query-API filters + `HighlightMetadata` + a **human correction/review loop** feeding the golden set via the `annotation` registry + `candidate_segments`
P5	**Owned generative video-QA model** (§5A)	global flywheel **+ per-user adapters**; P4 metadata is training signal ? fine-tune an open VLM (Qwen-VL) via LoRA ? serve as `owned_vlm` **provider with Gemini paid fallback**; extend the eval harness to QA quality
P6 *(visionary)*	**Gait / biomechanics under a supervising coach**	deliberately **back-burner**; technique feedback; highest Art. 9 bar ? gate behind explicit biometric + parental consent (§7)

P0?P1 deliver the headline ask (per-user upload + stored highlights + history). P2?P3 deliver the "plug-and-play storage/compute" ask. **P4 (rich semantic metadata) is the immediate next value stage once rally detection + highlights are solid** ? e.g. *"rallies that ended in a smash or backhand drive,"* *"rallies where the court was open but the player made an unforced error."* **P5** owns the model; **P6 (gait) is the visionary horizon, intentionally deferred.**
Sequencing principle (owner directive): make the scaffolding (P0?P4) **rock-solid first**, keep the **known paid stack as an always-ready fallback**, and only then shift focus to **owned-model quality** (P5). The provider registry keeps the paid path one config flip away.

7. Privacy & legal (gait raises the bar ? design for it now)

- **The load-bearing nuance:** biometric data is special-category **only when used to *uniquely identify* a person** (GDPR Art. 9). **Highlight/shot detection on players a coach already knows is general analytics ? generally *not* Art. 9.** But **gait/biomechanics that builds a per-athlete identifying template tips into Art. 9** and needs **explicit, granular consent** referencing biometric data. Engineer gait with that boundary (consent gate, purpose limitation, no cross-account identification); decide whether it runs **locally/on-tenant only** (strong posture) vs our cloud. (EDPB Guidelines 3/2019 on video devices apply.)
- **Minors (e.g. academies).** Parental/guardian consent and safeguarding are first-order, not an afterthought. Model consent explicitly per relationship (user?subject, coach?athlete where the overlay applies, parent?minor).
- **BYO-storage is the central privacy/liability lever:** if raw video stays in the tenant's own bucket/Drive and we process via signed URL or their compute, we minimize what we hold ? shrinking breach surface, residency, and erasure obligations.
- **Paid-LLM no-train (C4):** use **Gemini via Vertex AI** (contractual no-training), **not the consumer/free tier**; set short/zero retention, document it in the DPA, and surface "your footage is never used to train AI" as a trust feature for academies. For the gait phase, prefer keeping biometric inference off third-party APIs entirely.
- **Training consent is a distinct, higher bar (for the owned model, §5A).** Training on user video needs a **separate, explicit, opt-in training licence** ? not bundled with processing consent ? and it **compounds** the biometric (Art. 9) and minor-consent issues above. Train **only on explicitly contributed/licensed video**; never train on what merely passes through a tenant's bucket. **Exclude minors' video from the training

pool by default**, keep the training corpus a **separate, firewalled, opt-in sub-pool**, and log **per-clip consent/provenance** for revocation + audit. Incentives must not make consent non-free (the product stays fully usable without contributing). And the **teacher-distillation rule** (§5A): never train on paid-LLM outputs. **Two consent tiers (§5A):** *(a)* "improve **my own** detection with **my** data" ? a per-user adapter trained only on the user's own video (lowest friction; no cross-user mixing; deletion = drop the adapter); *(b)* "contribute to the **shared** model" ? pooled corpus, higher bar. Default for both is **off**.
- **Right to erasure / retention:** per-tenant, per-athlete delete must cascade across DB + storage artifacts (the `ON DELETE CASCADE` discipline already in `database.py` extends here).

8. Key decisions for the owner (the forks)

1. **Hosted-first vs BYO-first.** Recommended: **hosted control-plane first** (P0?P1) with **BYO-storage soon after** (P2); **BYO-compute later** (P3) as a power-user/enterprise option ? it's the highest-complexity, highest-security item.
2. **Database now or later.** Move to **Postgres at P1** for the hosted path, or stay SQLite-per-tenant if self-hosted-single-academy is the first GTM. (Abstract DB access regardless.)
3. **Auth: build vs buy** (Auth0 / Clerk / Supabase Auth / Cognito).
4. **Orchestration tech** (see research recommendation) ? bias to the simplest that gives retries + status.
5. **Gait processing location** ? local/on-tenant (privacy-max) vs cloud (capability-max).
6. **How far to push BYO-compute** given its security/ops complexity.
7. **Owned-model base + licensing (§5A).** Which commercial-OK video VLM to build on (Qwen-VL Apache-2.0 vs Gemma 3 custom license vs ?) and confirm a gated commercial derivative is permitted ? see COMMERCIALIZATION C14.
8. **Training-data sourcing & consent.** Opt-in contribution terms, incentives, and the teacher-distillation constraint (whether paid-LLM outputs may seed the corpus).
9. **When to start the owned model** ? strictly after the scaffolding + flywheel are solid (the explicit directive), with the paid provider as the standing fallback.

Risks: scope explosion (sequence strictly P0?P5); multi-tenant data-leak bugs (RLS + tests); BYO-compute security; **biometric/minor liability** (get this right before any gait feature ships); **training on user video without proper opt-in/licence**, and **base-model / teacher-distillation licence violations**; cost of hosted GPU (training *and* inference); **premature model-building** before data/eval are ready (mitigated by keeping the paid fallback and gating P5 behind the flywheel).

9. Sources

External best-practices research (2026-06-07) backing the recommendations above. Each underlying claim in the research was tagged fact-vs-synthesis; figures are as-of and should be re-validated before committing budget.

Storage abstraction: [fsspec](https://fsspec.com/) · [cloudpathlib](https://github.com/drivendataorg/cloudpathlib) · [smart_open](https://github.com/piskvorky/smart_open) · [Apache Libcloud](https://libcloud.apache.org/) · [gdrive-fsspec](https://github.com/fsspec/gdrive-fsspec) · [dropboxdrivefs](https://github.com/fsspec/dropboxdrivefs) · [AWS S3 presigned URLs](https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html) · [token-based S3 with STS](https://oneuptime.com/blog/post/2026-02-12-token-based-access-s3-sts/view) · [signed URLs across clouds](https://medium.com/@shaikhsarwan49/signed-urls-in-aws-gcp-and-azure-a-beginner-friendly-guide-e2e549425865)

Multi-tenancy: [Bytebase: multi-tenant DB patterns](https://www.bytebase.com/blog/multi-tenant-database-architecture-patterns-explained/) · [Hunchbite: RLS vs schema-per-tenant](https://hunchbite.com/guides/multi-tenant-saas-architecture) · [PlanetScale: tenancy in Postgres](https://planetscale.com/blog/approaches-to-tenancy-in-postgres) · [DEV: DB-per-tenant vs shared schema](https://dev.to/young_gao/multi-tenant-architecture-database-per-tenant-vs-shared-schema-1n2e)

BYO-compute / orchestration: [Pinecone BYOC](https://www.pinecone.io/blog/byoc/) · [Nuon: control/data-plane](https://nuon.co/blog/byoc-control-plane-data-plane-architectures) · [GitHub self-hosted runners (ARC)](https://docs.github.com/en/actions/hosting-your-own-runners/managing-self-hosted-runners-with-actions-runner-controller/about-actions-runner-controller) · [SkyPilot](https://github.com/skypilot-org/skypilot) · [SkyPilot AI control plane](https://blog.skypilot.co/ai-job-orchestration-pt2-ai-control-plane/) · [Modal (AWS Marketplace)](https://aws.amazon.com/marketplace/pp/prodview-j727623xqhh2k) · [Northflank: Anyscale alternatives](https://northflank.com/blog/anyscale-alternatives-for-ai-ml-model-deployment) · [DevPro: Python task queues 2025](https://devproportal.com/languages/python/python-background-tasks-celery-rq-dramatiq-comparison-2025/) · [Judoscale: choosing a task queue](https://judoscale.com/blog/choose-python-task-queue) · [Hatchet vs Temporal](https://hatchet.run/versus/hatchet-vs-temporal) · [Hatchet](https://github.com/hatchet-dev/hatchet)

Privacy / legal: [GDPR Art. 9](https://gdpr-info.eu/art-9-gdpr/) · [GDPR Local: biometric compliance](https://gdprlocal.com/biometric-data-gdpr-compliance-made-simple/) · [Legiscope: Art. 9](https://www.legiscope.com/blog/gdpr-article-9-special-categories.html) · [EDPB Guidelines 3/2019 (video)](https://www.edpb.europa.eu/sites/default/files/files/file1/edpb_guidelines_201903_video_devices_en_0.pdf) · [Fox Rothschild: EDPB video guidance](https://www.foxrothschild.com/publications/video-surveillance-under-gdpr-edpb-issues-final-guidance) · [ICO: children & UK GDPR](https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/childrens-information/children-and-the-uk-gdpr-old/what-are-the-rules-about-an-iss-and-consent/) · [EDPB Guidelines 05/2020 (consent)](https://www.edpb.europa.eu/sites/default/files/files/file1/edpb_guidelines_202005_consent_en.pdf) · [AI data-retention compared](https://ax-sentinel.com/blog/ai-data-retention-policies-compared) · [Xenoss: enterprise LLM platforms](https://xenoss.io/blog/openai-vs-anthropic-vs-google-gemini-enterprise-llm-platform-guide)

Comparable products: [Veo](https://www.veo.com/) · [Veo Help Center](https://support.veo.com/hc/en-us/articles/4459159526929-Overview-of-cam-veo-co-Record-manage-and-upload-footage-with-your-Veo-Cam-1) · [Latterly: Hudl competitors](https://www.latterly.org/hudl-competitors/) · [Apple: Behind the Design ? SwingVision](https://developer.apple.com/news/?id=0pg4dthn) · [SwingVision FAQ](https://swing.vision/faq) · [Sportsvisio: coach video-analysis comparison](https://www.sportsvisio.com/stories/video-analysis-software-for-coaches-complete-comparison)

Training backends & harness (§5A): [karpathy/nanochat (harness reference)](https://github.com/karpathy/nanochat) · [HF nanoVLM (minimal VLM)](https://github.com/huggingface/nanoVLM) · [nanoVLM blog](https://huggingface.co/blog/nanovlm) · [Tinker (GA + vision)](https://thinkingmachines.ai/blog/tinker-general-availability/) · [Tinker model lineup](https://tinker-docs.thinkingmachines.ai/model-lineup) · [Tinker ToS (weights/no-train)](https://thinkingmachines.ai/legal/tos/) · [Fireworks VLM fine-tuning](https://fireworks.ai/blog/vlm-tuning) · [Together AI fine-tuning](https://www.together.ai/fine-tuning) · [Predibase: fine-tune & serve VLMs](https://predibase.com/blog/how-to-fine-tune-and-serve-vlms-in-predibase) · [Predibase RFT/GRPO](https://docs.predibase.com/user-guide/fine-tuning/grpo) · [ms-swift (native video/audio)](https://github.com/modelscope/ms-swift) · [LLaMA-Factory (VLMs)](https://github.com/hiyouga/LlamaFactory) · [Unsloth (Qwen3-VL)](https://qwen.readthedocs.io/en/latest/training/unsloth.html) · [Fine-tuning tools 2026 (LoRA/QLoRA)](https://www.index.dev/blog/top-ai-fine-tuning-tools-lora-vs-qlora-vs-full)

Owned model & licensing (§5A): [Qwen2.5-VL](https://qwenlm.github.io/blog/qwen2.5-vl/) · [Qwen2.5 license split](https://qwenlm.github.io/blog/qwen2.5/) · [Qwen3-VL](https://github.com/QwenLM/Qwen3-VL)

· [Gemma 3 (multimodal)](https://huggingface.co/blog/gemma3) · [Gemma Terms of Use](https://ai.google.dev/gemma/terms) · [Gemma license-risk analysis](https://wcr.legal/google-gemma-license-risks/) · [Gemma 4 / Apache-2.0](https://www.mindstudio.ai/blog/what-is-gemma-4-apache-2-license-commercial-ai-deployment) · [Llama 3.2 license (700M MAU + naming)](https://www.llama.com/llama3_2/license/) · [gpt-oss (text-only, Apache-2.0)](https://openai.com/index/introducing-gpt-oss/) · [Gemini API terms (compete clause)](https://ai.google.dev/gemini-api/terms) · [OpenAI terms](https://openai.com/policies/row-terms-of-use/) · [Anthropic: outputs & training](https://support.claude.com/en/articles/12326764-can-i-use-my-outputs-to-train-an-ai-model) · [QLoRA practical](https://apxml.com/courses/fine-tuning-adapting-large-language-models/chapter-4-parameter-efficient-fine-tuning/practice-qlora-fine-tuning) · [VLM LoRA cookbook](https://huggingface.co/learn/cookbook/en/fine_tuning_vlm_trl) · [GDPR Art. 9](https://gdpr-info.eu/art-9-gdpr/) · [athletes & biometric data](https://www.architectureandgovernance.com/applications-technology/biometric-data-and-athletes-privacy-law-and-compliance-implications/) · [federated learning + DP](https://research.google/blog/federated-learning-with-formal-differential-privacy-guarantees/) · [self-hosted LLM VRAM/cost](https://www.gpu-mart.com/guides/self-hosted-llm) · [Gemini API pricing](https://ai.google.dev/gemini-api/docs/pricing)

See also
- [`COMMERCIALIZATION.md`](COMMERCIALIZATION.md) ? C12 (SaaS foundations), C4 (paid-LLM), economics.
- [`DEFERRED_HARDENING_PLAN.md`](DEFERRED_HARDENING_PLAN.md) ? #16 async job model (the P0 infra slice), #7 packaging.
- [`PMF_MARKET_RESEARCH.md`](PMF_MARKET_RESEARCH.md) ? academy/coach beachhead (who this serves).
- [`CODE_MAP.md`](CODE_MAP.md) ? current architecture + the registry extension points.
- [`ROADMAP.md`](ROADMAP.md) ? offline self-improving rally detection (the pipeline's own trajectory).

6. user (2026-06-08T13:03:36.139Z)

Deferred Hardening Plan ? Train/Eval Guardrails, Async Jobs, Packaging

Status: ? Design for review ? **no code yet**. Owner-requested follow-up to the
"config & identity consistency" tech-debt pass. Each item below becomes its own PR
only after this doc is approved.

Why these three: they were cataloged as out-of-scope during the consistency
pass because each is larger/design-heavy or needs the owner's machine. They are the
remaining high-value hygiene items not already done (config/structure/Result/logging/
reports) or in flight (detector abstraction, PR #49).

Recommended sequence: #7 Packaging (lowest risk, unblocks clean installs/CI) ?
#13 Train/eval guardrails (correctness of the data flywheel) ? #16 Async jobs
(largest, product-facing).

---

1. Packaging modernization (debt #7)

Problem
- No `pyproject.toml` on `master`. (PR #49's description lists packaging as "done",
  but it isn't present ? reconcile with the owner.)
- The root `setup.py` is a **diagnostics/dependency-checker**, not a PEP 517 build
  script ? its name collides with setuptools semantics and will confuse `pip`/`build`.
- Deps live only in `requirements.txt`; `obsreport` is a commented soft dep
  (BACKLOG) and `supervision<0.30.0` is pinned (debt #14).

Approach
- Add `pyproject.toml` (PEP 621): project metadata + runtime deps mirroring
  `requirements.txt`, with optional-dependency groups:
  - `[project.optional-dependencies] dev` ? `pytest`, coverage tooling.
  - `reporting` ? `obsreport` pinned to its git tag (`v0.1.3`, per BACKLOG).

  - `gpu` ? torch/torchvision (note: the cu124 `--extra-index-url` is **not** expressible in standard PEP 621 deps; document the install line or use a `[tool.uv]`/pip-conf approach rather than faking it).
- Console entry points: `indexer = backend.main:main` (so `indexer --server` works), optionally eval drivers.
- **Rename the diagnostic `setup.py`** (e.g. `scripts/doctor.py`) so it no longer shadows the build system; update README/`run_all_tests.py`/CI references.
- Choose a build backend (hatchling recommended) and confirm the `backend/` import root stays valid (keep current layout; no src/ move required).

Risks
- `setup.py` rename touches docs and any muscle-memory invocations ? grep first.
- Torch CUDA wheels can't be pinned cleanly in PEP 621; must document the `--extra-index-url` install path (the current `setup.py` logic can move into docs).
- CI must switch to `pip install -e .[dev]`.

Verification
- `pip install -e .[dev]` in a clean venv ? `run_all_tests.py` green.
- `indexer --help` resolves via the entry point.

---

2. Train/eval separation guardrails (debt #13)

Problem
Separation is **by convention only**. `backend/eval/calibrate_local.py` and `calibrate_wasb.py` do honest leave-one-video-out (LOVO), but nothing *enforces* that a video used to train the learned segmenter (`backend/eval/training.py::build_training_set` ? `distill_local`) is absent from the eval/regression set. Leakage would silently inflate the very metrics the product story rests on (? in BACKLOG).

Approach
- **Split unit = match, never clip** (per `GOLDEN_SET_PHASE1_VIDEOS.md`: split by match, hold ~? of each sport as eval).
- Add a `split: "train" | "eval"` field to the collector's `curate export` manifest (cross-repo contract, ADR-002 lineage) ? or a separate eval manifest.
- New `backend/eval/splits.py`:
  - `load_split(manifest) -> (train_ids, eval_ids)` keyed by **match id** (not just file MD5 ? re-encoded duplicates share a match but differ in `compute_video_id`, so group at match level).
  - `assert_disjoint(train_ids, eval_ids)` ? raise on any overlap.
- Wire the guard into the training entrypoint (`distill_local`/`training`) and into `run_regression.py` so the baseline is computed strictly on the eval split.

Risks
- Requires a collector-side manifest change (coordinate the cross-repo contract).
- Small corpus ? tiny held-out set ? high metric variance; the guardrail is correct but the numbers will be noisy until the golden set grows.
- MD5-only dedup is insufficient (re-encodes evade it) ? must track match identity.

Verification
- Unit test: overlapping train/eval ids ? `assert_disjoint` raises.
- Integration: a deliberately-leaked manifest fails the training entrypoint.

---

3. Async job model (debt #16)

Problem
`POST /api/process` runs validate ? segment ? report **inline**, blocking the request for the full duration of a long video. Unworkable as a hosted/product API.

Approach (minimal first slice ? single self-hosted box)
- `POST /api/process` returns **202 + `job_id`**; add `GET /api/jobs/{job_id}`

```
  returning `{status, progress, result, reports}`.
- Run the pipeline on a background worker ? start with a `ThreadPoolExecutor`
  (segmenters already use threads internally for AI handoff, so this fits); leave a
  real queue (RQ/Celery) for later if multi-process is needed.
- Persist job state in the existing SQLite DB via the migration ladder
  (`backend/infrastructure/database.py::_init_db`, bump `PRAGMA user_version` 1 ? 2,
  add a `jobs` table). The observability report becomes the job's artifact.
- Update the static UI to poll `GET /api/jobs/{id}` instead of blocking.

### Risks
- **SQLite thread-safety:** verify `check_same_thread`/connection-per-thread;
  `_get_connection` already re-issues `PRAGMA foreign_keys=ON` per connection ?
  confirm it's safe under the executor.
- Cancellation/timeout and crash recovery (a job in RUNNING after a restart).
- In-process store is not horizontally scalable ? acceptable for the current
  single-box self-hosted target; call it out so it isn't mistaken for production-HA.

### Verification
- Submit a job ? poll ? DONE with the same segments the sync path produced.
- Server restart mid-job leaves a recoverable/clearly-FAILED job, not a hang.

---

## Out of this doc
- **Detector/WSL abstraction (#6):** in flight (PR #49).
- **Unify hybrid segmenters (#7 in LOW_REGRET):** owner-deferred.
- **ByteTrack `supervision<0.30.0` migration + real-footage validation (#14/#15):**
  need a GPU/WSL box and real video ? owner-machine work, tracked separately.
```

7. assistant (2026-06-08T13:03:38.100Z)

Now let me read the remaining documentation files.

8. user (2026-06-08T13:03:39.024Z)

```
# Architecture Decision Records

## ADR-001: Precise shuttle tracking via self-hosted TrackNetV4/WASB on WSL2

**Date:** 2026-05-25
**Status:** Accepted

### Context

The pipeline uses an AI multimodal provider (Gemini) for *semantic* rally segmentation.
We need **precise frame-level detection of the moving shuttlecock** (and, generically,
balls for other sports) to produce higher-quality candidate rallies and a fine-tuning
signal.

We evaluated whether a commercially available model/API could do this. Findings:

- **Multimodal LLM APIs (Gemini, Claude, GPT-class) are the wrong tool for precise
  localization.** They reason well about *what* is happening (rally vs. dead time) but
  cannot reliably track a tiny, fast (~up to 400 km/h), motion-blurred shuttle frame by
  frame.
- The precision leaders are **purpose-built trajectory models**, which are open source and
  self-hosted, not big-vendor APIs:
  - **TrackNetV4** (TensorFlow) ? badminton-specific, heatmap + motion-attention,
    highest reported shuttle accuracy; ships `src/predict.py` producing per-frame (X, Y)
    coordinate CSV. https://github.com/AR4152/TrackNetV4
  - **WASB** (PyTorch) ? multi-sport strong baseline (badminton/tennis/volleyball/?),
    fits our `SportProfile` abstraction; needs a custom-video inference wrapper.
    https://github.com/nttcom/WASB-SBDT
```

- True turnkey commercial APIs (Roboflow hosted models) exist but cap below a tuned TrackNet; enterprise systems (Hawk-Eye, SkillCorner, TRACAB) are not API-accessible.

Decision

1. Adopt **self-hosted TrackNetV4** as the primary precise shuttle detector, with **WASB** as the multi-sport fallback.
2. Run these models under **WSL2 (Ubuntu)**, **not** native Ubuntu. The entire indexer already runs on Windows; WSL2 keeps a single dev environment and quarantines the TensorFlow/PyTorch dependencies. Native Ubuntu would require dual-boot and migrating the whole project for marginal gain.
3. Integrate via a **subprocess adapter**: a detector module that shells out to the WSL `predict.py`, reads the coordinate CSV, and converts trajectory ? candidate rallies feeding the existing stitching flow. This keeps the TF dependency fully isolated from the main Python environment.

Consequences

- **WSL I/O caveat:** `/mnt/c` access is slow. Stage match videos inside the WSL filesystem (e.g. `~/clips/`) before inference rather than reading large mp4s across the Windows?WSL boundary.
- TrackNetV4 (TensorFlow) and WASB (PyTorch) live in **separate conda envs**.
- Next step after integration: manual video annotation ? fine-tune the detector.

ADR-002: Use the sports-data-collector's rally annotations as a golden eval + temporal-segmenter training set

Date: 2026-05-26
Status: Accepted

Context

A sibling project, **sports-data-collector**, curates sports videos and stores per-rally `[start, end)` timestamps (plus stroke counts, scores) in an SQLite DB, with a commercial-license flag per source. We asked whether this data could improve the indexer's rally detection, and for which task.

Key findings:

- The annotations are **temporal-only** (when rallies start/end). They do **not** contain per-frame shuttle `(x, y)` coordinates, so they **cannot train the shuttle detector** (TrackNetV4/WASB ? see ADR-001), which needs coordinate labels.
- They are an excellent **golden evaluation set** and a **training signal for the *temporal* rally segmenter** (active vs. dead play over time) ? the stage that is today a hand-tuned heuristic, not a learned model.
- The collector's five sports tables (`badminton_rallies`, `tennis_points`, `cricket_deliveries`, `pickleball_rallies`, `tabletennis_rallies`) all share a `(video_id, ordinal, start_time, end_time)` shape, and the indexer already has `sport_registry` / `segmenter_registry`. So a *generic* `(start, end)` bridge serves every sport.
- The two repos key videos differently: the collector by a human `video_id`, the indexer by **file MD5**. A re-encode/re-download changes the bytes, hence the MD5.

Decision

1. Treat the collector's rally timestamps as (a) the **golden eval set** for rally *segmentation* and (b) the **labels** for training a temporal segmenter ? explicitly **not** as shuttle-detector training data.
2. Bridge the repos with a sport-agnostic **export** (`curate export` in the collector):

per-video `start,end` CSVs (the indexer's eval format) + a `manifest.json` that pairs each CSV with its video file. Gate on `is_commercial` so only commercially usable data flows through.
3. Honour the **MD5 keying** contract: evaluation/processing must run on the exact video file recorded in the manifest; acquisition (ADR-003) therefore precedes processing.

Consequences

- The collector becomes the *producer* of labels; the indexer the *consumer* (processing + scoring + training). Neither takes a hard dependency on the other beyond the manifest.
- Scoring is free and deterministic (reads the DB; no API). See `docs/EVALUATION.md`.
- Only invest in frame-level coordinate labels (for the detector) **if** evaluation shows *detection*, not *segmentation*, is the bottleneck.

ADR-003: Video acquisition ? always best resolution, upgrade in place, authenticated cookies

Date: 2026-05-26
Status: Accepted

Context

The collector's initial crawl downloaded videos with `yt-dlp -f worstvideo` (256×144), mislabelled as 1080p. The indexer needs ?720p MP4 for meaningful YOLO/shuttle detection. Re-acquiring full-resolution sources surfaced several hard-won realities:

- The ShuttleSet corpus **tops out at 1080p** (no 4K); "best available" resolves to 1080p.
- Authenticating with a **YouTube Premium** account is required for reliable, unthrottled downloads ? anonymous downloads are throttled (~1.3 MiB/s) and **non-deterministic** (identical commands variously yielded 1080p DASH, 1080p HLS, or a 144p fallback).
- **Chrome** cookies can't be read on Windows (app-bound encryption / DPAPI, yt-dlp #10927); **Firefox** cookies read cleanly even while open.
- The `web`/`web_safari` player clients are forced into **SABR streaming**, which strips download URLs and silently drops to 144p (yt-dlp #12482). yt-dlp's `default` client serves real downloadable 1080p DASH with auth cookies.
- Bulk bursts trip YouTube **rate-limiting** (a 22-video burst failed all at once, though each worked in isolation).

Decision

1. **Always fetch the best resolution available** (`bv*+ba/b -S res,...`, merged to MP4); ffprobe the result and store the *actual* width/height/fps ? never assume.
2. **Upgrade in place, never clean-slate:** the `fetch` command re-downloads a video and updates only `local_path`/`resolution`/`fps`, preserving every rally annotation and curation status (deleting would cascade-delete the valuable labels).
3. Use **authenticated cookies** (`--cookies-from-browser firefox` / `--cookies file`) and the **`default` player client**; enable the EJS challenge solver (deno).
4. **Throttle bulk runs** (`--sleep` + per-video `--retries`/backoff) and treat a below-min-height download as a **failure** (discard, don't commit) so a flaky low-res result never overwrites a good entry.

Consequences

- All 22 fetchable ShuttleSet matches are now true 1080p (one match, shuttleset_12, has no

  source URL in the catalog and remains video-less but keeps its annotations).
- Acquisition is reproducible and resumable; the ~26 GB corpus is kept outside git.
- Future non-YouTube or 4K sources are supported by the same code (optional `max_height`).

ADR-004: Add an offline, API-free *learned* rally segmenter as a first-class mode

Date: 2026-05-26
Status: Proposed (substrate pending data)

Context

The default high-quality path (`yolo_hybrid`) refines YOLO candidate windows with **Gemini**,
which costs money and requires network access. With a now-1080p, commercially-licensed,
rally-annotated corpus (ADR-002/003), we can learn the refinement step offline instead.

Crucially, the **API-free signal already exists and is already persisted**: `yolo_hybrid`
Stage 1 is pure YOLO (player tracking ? per-window motion features: peak/mean velocity,
active-frame count, track-id count) stored in `candidate_segments.stats`. Gemini is only
the Stage-2 refiner. So the rally annotations can **train a classifier to replace Gemini**,
with no external API and **no new labeling**.

Three candidate substrates were weighed:

- **Tune `motion` thresholds** on the annotations ? trivial, fully offline, but a low
  ceiling (pixel motion can't separate rally from crowd/pan/replay). Use as a *floor*.
- **YOLO features ? trained classifier** ? reuses already-stored, semantically meaningful
  player-motion features; medium-high ceiling; moderate effort. *Recommended primary.*
- **WASB shuttle-motion ? rally detector** ? highest ceiling (the shuttle defines play),
  fully offline but heaviest (needs the WSL2 WASB runner, ADR-001); can't train WASB
  itself (no xy labels), only a rally layer on its output.

Decision

1. Build an **offline, API-free learned segmenter** as a new registry mode (e.g.
   `yolo_learned`): label detector windows by overlap with GT rallies, train a lightweight
   classifier on the persisted features, emit final segments with **zero API calls**, and
   score it with the existing harness against `motion` and a Gemini reference.
2. **Choose the substrate from data, not assumption.** A learned refiner can only sharpen
   the windows the detector found, so the deciding metric is the detector's **candidate-stage
   recall** vs. the annotations: high recall ? build on YOLO features; low recall ? bring in
   WASB's shuttle signal to raise the ceiling.
3. Keep the Gemini-backed `yolo_hybrid` as an **upper-bound reference**, not a production
   dependency.

Consequences

- The final tool gains a **self-contained, offline, free** rally mode trained on its own
  curated data ? runnable air-gapped, improvable as more matches are annotated.
- The approach is **sport-generic**: the same label?feature?classifier machinery extends to
  tennis/cricket once those have annotations.
- The full paid 22-video Gemini evaluation is **not** run; spend is limited to a single

  reference video.

Update (2026-05-26): Phase 3 evidence ? substrate is WASB, not YOLO

The deciding candidate-recall measurement is in. On shuttleset_8 (64 GT rallies), pure
YOLO Stage-1 produced 181 candidate windows with:

- **Recall @ IoU 0.5 = 0.05**, but recall @ IoU 0.1 = 0.53 and @ IoU 0.01 = 0.72.
- **72%** of rallies have *some* candidate overlap, yet the **median best coverage of a
  rally by any candidate is only ~20%**.
- Post-hoc **merging adjacent candidates does not help** (recall stays ~0.05 and degrades
  as the merge gap grows ? windows overshoot rather than align).

Interpretation: YOLO isn't *blind* to rallies, but its windows are **badly misaligned** ?
because the **broadcast camera tracks the players**, their on-screen (pixel) velocity dips
during rallies and spikes on pans/cuts/replays. Player-pixel-motion is therefore a poor
rally proxy on broadcast footage (it was designed for *static* GoPro recordings). A
classifier that only *filters* these windows is capped at ~0.25 recall ? below any useful
bar ? and no simple boundary post-processing recovers it.

Decision update: per this ADR's data-driven rule (low candidate recall ? bring in the
shuttle signal), the offline segmenter's substrate is **WASB shuttle-motion** (ADR-001),
not YOLO player-motion. The shuttle's motion/presence is camera-invariant and directly
defines active play. The `motion` segmenter is likewise unusable here (0 TP ? moving-camera
pixel motion is everywhere).

Caveats / next: this is one video; the mechanism (camera tracking) generalizes but
should be confirmed on 2?3 more via a *free, local, candidates-only* YOLO pass (no Gemini).
Processing is also slow (~50 min per 1080p video) ? the detection pass likely needs reduced
resolution and/or cached detector outputs to be practical across 22 matches. The
substrate-agnostic training-data + classifier layers already built still apply: they will
consume **WASB-derived** windows/features instead of YOLO's.

ADR-005: Permissive Dependency Stack for the Hosted Monetization Path

Date: 2026-05-30
Status: Accepted

Context

The project will be monetized as a **closed-source hosted API** (users upload videos to a cloud GPU VM and get highlights). Because we never distribute code/binaries, GPL/LGPL distribution copyleft is not triggered ? but **AGPL §13** (network clause) and **metered-API terms** are, and **video-codec patent royalties** are a separate concern from software licenses. A deep-research audit (2026-05-30, 25 adversarially-verified claims against primary sources ? see `docs/MONETIZATION_AUDIT.md`) identified the blockers and clean alternatives. This ADR also **resolves the open licensing question in ADR-001/ADR-004** about WASB.

Decision

1. **Drop `ultralytics`.** Both the library and the `yolov8n.pt` weights are **AGPL-3.0**; §13 makes them a blocker for a closed-source hosted service. Swapping the codebase while keeping the `.pt` does **not** escape ? the weights are AGPL too. Replace player/motion-

gating detection with a permissive detector: **torchvision Faster R-CNN / RetinaNet (BSD, default ? zero new deps, person = COCO class 0)**, escalating to **YOLOX** or **RT-DETRv2 (Baidu repo, Apache-2.0)** if more speed/accuracy is needed. **Ultralytics Enterprise License** is a paid fallback only (quote-only pricing).
2. **Shuttle tracking stays permissive.** **WASB-SBDT and TrackNetV3 are MIT (CLEAR for commercial use)** ? this resolves the licensing unknown flagged in ADR-001/ADR-004. **MonoTrack (Adobe Research, noncommercial) and YOLO-NAS weights (Deci, noncommercial) are banned.** Train our own checkpoints (or confirm per-checkpoint terms), since distributed research weights/datasets aren't covered by the MIT *code* grants.
3. **LLM step ? keep all three strategies open:** (A) eliminate via an offline temporal segmenter (**ActionFormer, MIT** ? the Phase-4 direction), (B) self-host **Qwen2.5-VL-7B/32B (Apache-2.0)** or **InternVL2.5 (MIT)**, or (C) keep **Gemini 2.5 Flash on the paid tier** (required ? the free tier trains on and human-reviews uploaded video). The `google-genai` SDK itself is Apache-2.0 (clear).
4. **Emit royalty-free output.** Standardize highlight output on **AV1 (+ Opus)** to avoid AVC/HEVC encode-side patent exposure; provision an **Ada-class GPU (L4/L40)** for hardware AV1 (NVENC), falling back to **SVT-AV1 (CPU, BSD-3-Clause-Clear)** on T4/A10/A100.

Consequences

- Eliminates the only two hard legal blockers; the rest of the stack (torch, opencv, fastapi, numpy, lapx, ffmpeg-as-subprocess, google-genai SDK) is already permissive.
- Detector swap is contained to `backend/indexer/yolo_hybrid.py` (the `from ultralytics import YOLO` load + the `model.track(...)` call) plus `config.json` knobs and `tests/test_yolo_hybrid.py` mocks. Stage-1 feature outputs (velocity/active-frame/track-id stats) and Stages 2?4 are detector-agnostic and unchanged.
- We now own the frame loop + tracking (ultralytics did both internally). Use a permissive ByteTrack (e.g. `supervision`, MIT, or the MIT ByteTrack repo) on top of `lapx` (already a BSD dep).
- Residual items needing legal sign-off: exact codec-pool royalty thresholds (AVC/HEVC decode of uploads), Ultralytics Enterprise pricing, and per-checkpoint weight licenses ? tracked in `docs/MONETIZATION_AUDIT.md` §7.
- The `yolo_hybrid` registry key is now a slight misnomer (no YOLO); keep it for back-compat or rename in a follow-up.

ADR-006: Golden-Set Generation via Vendor Bake-off + a Pluggable Annotation Provider

Date: 2026-05-30
Status: Accepted (process v1; vendor shortlist pending research)

Context

The product needs **golden-set ground truth** (hand-verified rally `[start,end)` boundaries) to (a) measure pipeline quality and (b) run **on-demand / surprise regression tests** against the production tool. Two forces shape the design: the hosted-API monetization goal raises a "who views the user's video" concern (ADR-005 / MONETIZATION_AUDIT.md), and the founder is **Bangalore-based** and prefers **local/India vendors**. We already own a deterministic interval scorer (`backend/eval/metrics.py`) and a growing collection of **licensed/owned** match videos.

Decisions locked with the owner: **licensed/owned video only** (no user uploads to vendors) · **rally `[start,end)` annotation depth** · **tiered turnaround** (fast automated + slow human) · **vendor bake-off** to choose · **Bangalore/India-local preference**.

Decision

1. **One scorer, three jobs.** A vendor's `[start,end)` output is structurally identical to a prediction, so `metrics.py` grades product-vs-truth, vendor-vs-key, and annotator-vs-annotator (IAA) with no new scoring code.
2. **Select vendors by measured quality-per-dollar**, via a bake-off: write a precise rally-definition guideline, build a small internal gold key (5?10 licensed clips), send the same clips to 2?3 vendors (favoring Bangalore/India), and grade every submission with our own harness. The numbers are the decision.

3. **Pluggable `AnnotationProvider` registry** mirroring the existing registry pattern: `FileProvider` (today's manual/ShuttleSet flow), `HumanVendorProvider` (slow tier ? authoritative truth), `AutomatedProvider` (fast tier ? independent model oracle).
4. **Regression = existing harness + one new component.** `regress(video)` calls a provider for ground truth, runs the production pipeline, and scores with the unchanged `metrics.py` against a per-video `baseline.json`.
5. **Privacy by construction.** Golden sets use only licensed/owned footage; all third-party video egress flows through the single provider boundary where DPA / no-reuse / delete-on-delivery / access-logging controls live.

Consequences

- Eliminates the "human views user video" concern for golden-set/regression work, because user uploads never enter the pipeline.
- **The oracle problem is a first-class caveat:** the fast-tier `AutomatedProvider` must be a *different model lineage* than production (else shared blind spots make the test falsely green); fast-tier alarms escalate to the slow human tier before blocking a release.
- New docs: `docs/GOLDEN_SET_VENDORING.md` (the full process v1) and `docs/VIDEO_RECORDING_GUIDELINES.md` (a tracked draft: capture-quality thresholds to be derived empirically from precision/recall, since input quality bounds output quality ? cf. ADR-004's broadcast-camera finding).
- The vendor shortlist itself is **pending a primary-source research pass** (favoring India/Bangalore) and is intentionally left blank until verified ? no un-sourced vendor claims.
- **Honest limitation:** a licensed/owned corpus may not fully represent real user capture, so regression greens are strong evidence, not guarantees ? the recording-guidelines work aims to narrow that gap.

Update (2026-06-02): shortlist researched; Roora selected for the initial pilot

- The **vendor shortlist** is filled in (GOLDEN_SET_VENDORING.md §6, primary-source pass 2026-05-30).
- The founder selected **Roora ML Pvt Ltd** (HSR Layout, Bengaluru) for the **initial paid pilot**, diverging from the doc's iMerit/Taskmonk default toward a Bangalore-local, startup-friendly shop. Roora's existence, image/video/temporal capability, and location are verified (rooragrp.com); **pilot quality is still to be measured** ? run as a **single-vendor pilot first** (the pilot *is* the calibration), not a multi-vendor bake-off.
- **Step 0 ? the annotation guideline ? is authored** (the highest-leverage artifact): collector repo `docs/annotation/` ? `ROORA_BRIEF.md` (rally CSV schema), `ANNOTATION_GUIDE.md` (illustrated guide), plus intake-manifest and NDA templates.
- **Pilot scope is 3 sports** (badminton, pickleball, table tennis); the collector's `import-local` ingestion for all three is implemented (collector ADR-001).
- **Still pending:** the internal **gold key** (5?10 expert-labeled clips) and the vendor-dependent **GS-3 `HumanVendorProvider`** ? GS-1/GS-2/GS-4 (FileProvider, regression runner, automated-oracle scaffold) have already landed (see GOLDEN_SET_IMPLEMENTATION.md).

ADR-007: Audio "thwack" hit-detection (fused with WASB) is the next rally-detection cue; pose/stance parked

Date: 2026-06-07
Status: Accepted (direction); experiments gated on golden eval labels

Context

After building the WASB shuttle-trajectory substrate (ADR-001), a Gemini-teacher ? local-student distillation (ADR-004 lineage), and measuring both, two honest findings emerged: **(a)** threshold-tuning the local windowing ceilings around F1 ? 0.44 and the learned student *underperforms* the threshold baseline at n=2 labeled videos; **(b)** the dominant failure is **candidate RECALL** ? WASB-derived windows simply don't align with (or miss) true rallies (IoU-0.5 recall ceiling ~0.43 on one clip). To decide whether teacher?student was even the right backbone, we ran a deep-research survey (`docs/RALLY_DETECTION_RESEARCH.md` ? 23 sources, 25 adversarially-verified claims). Verdict: **teacher?student is sound but NOT uniquely best**; the strongest direction is a **complementary, fully-local multi-cue stack**, and the **#1 cheapest lever is audio hit detection fused with the trajectory** (audio+visual fusion measurably lifts rally *recall* ? tennis HMM 83% fused vs 54% visual / 63% audio). Pose/court serve-gating is the validated #2 but is engineering-heavier.

Decision

1. **Adopt a fully-local AUDIO hit-detection cue** (shuttle "thwack" impacts) and **fuse it with the WASB trajectory** ? primarily to recover rallies WASB misses (recall) and secondarily as features for the distilled student. Detailed design: `docs/AUDIO_RALLY_DETECTION_PLAN.md`.
2. **Gate on an empirical GO/NO-GO probe (E1).** Shuttle impacts are quiet vs tennis/TT, so before building anything heavy, measure on *real GoPro audio* whether hit-clusters align with golden rallies and recover WASB misses. Build E2/E3 only if E1 passes.
3. **Keep Gemini as an OFFLINE teacher only** (never runtime) ? research confirms this is the correct role for a cloud VLM; it is not the deployed localizer.
4. **Park pose/court-geometry stance detection as the #2 cue** (serve-ready START + flight-gradient END). It is expected to be substantially harder to get right; revisit after audio (and a TBD owner hunch).
5. **Make "audio always enabled" a capture requirement** (`docs/VIDEO_RECORDING_GUIDELINES.md`), since the cue is only as good as the recording.
6. Stay permissive (ADR-005): classical DSP (numpy + stdlib `wave`, BSD), no restrictive weights; any later model trained in-house.

Consequences

- New `backend/pipeline/audio/` module (extraction + onset/peak + confidence), feeding `distill_local` and scored by the existing `metrics.py` / golden loaders. *(As built: the detector is **pure numpy + stdlib `wave`, no scipy** ? the anticipated scipy dependency proved unnecessary.)*
- **All audio measurement is blocked on golden eval labels** ? which the VLC rally-annotator (now a standalone MIT, multi-sport open-source repo: https://github.com/avidullu/rally-annotator) lets the owner self-produce, and the in-repo `--labels` loaders / collector `import-local` ingest.
- Sport-generic: tennis/table-tennis hits are *louder*, so the cue should transfer at least as well there.
- **Honest risk:** if E1 shows shuttle SNR is too poor on real amateur audio, audio is deferred ? this ADR commits to the *experiment and direction*, not to a guaranteed win.

Integration architecture: modular fusion now (Option A), an owned end-to-end model later (Option B)

A recurring design question: do we **feed audio into WASB** so it refines its own output, or **keep audio as a separate signal fused in our code**? The answer today is the latter ? and understanding *why* sets up the long-term route.

Why fusion lives in our rally layer, not inside WASB (Option A ? now). WASB is a

frozen, vision-only shuttle *detector* (image frames ? per-frame shuttle `(x, y, visibility)`; MIT weights we *run*, not train). It has no audio pathway and it does not decide rallies at all ? **rally detection is our own layer** on top of its trajectory (`trajectory_to_action_windows` ? gate ? the distilled classifier). Making WASB consume audio would mean bolting on an audio encoder and **retraining the network**, which needs per-frame shuttle-coordinate-*plus*-audio labels we don't have (we only have rally `[start,end]` labels) and would fork the clean MIT model. It also conflates two different questions: WASB answers ***where* the shuttle is**; audio answers ***when* a hit happened**. Audio doesn't make WASB localize better ? it's an independent rally-activity cue. So fusion belongs where cues are *combined into rallies*: our layer.

```
frames ?? WASB (frozen, MIT) ?? shuttle trajectory ??
audio  ?? hit detector (ours) ?? hit events ??????????? RALLY LAYER (ours) ?? rallies
[pose  ?? stance heuristic (ours; the parked #2) ????   (feature + decision fusion)]
```

Two complementary fusion modes (both in `docs/AUDIO_RALLY_DETECTION_PLAN.md`):
1. **Feature-level (primary):** audio features (hit_rate, inter-hit cadence/regularity, time-since-last-hit) join WASB-derived features (velocity, net-crossings) in the **single distilled classifier** that emits rallies ? "use audio to sharpen our rally metrics" (experiment E3).
2. **Decision-level recall-recovery:** where WASB proposed nothing but a dense hit-cluster exists, propose a rally there ? directly correcting WASB's recall misses.

Each cue (WASB, later the person/pose detector) stays a **swappable black box**; every signal is independently testable ? which matters while we're label-starved.

The long-term route (Option B ? the intended destination, once a healthy golden corpus exists). As we accumulate enough golden eval/training videos (owner self-rating via the annotator + vendor labels), the right end-state is to **collapse the modular cues into a single OWNED multimodal rally model**, trained/fine-tuned end-to-end on (frames + audio + pose/stance + ?) ? rally boundaries ? the HitNet / temporal-action-localization direction surfaced in `docs/RALLY_DETECTION_RESEARCH.md`. Crucially, that lets us **fine-tune from our own code and data instead of depending on WASB's pretrained weights** ? removing both the *recall ceiling* imposed by WASB's candidate windows and the *licensing question* on its academic-trained weights.

Why we don't jump to Option B now ? and why Option A is the path to it. Option B needs data we don't yet have, and it's far harder to debug as one black box. So we deliberately build **modular cues first** ? audio (this ADR), then a stance heuristic (the parked #2), then other signals ? each cheap, debuggable, label-efficient, and *each one generating the very golden labels an end-to-end model will need*. When the corpus is healthy, we train the unified model. **Option A is not a detour from Option B; it is how we earn it.**

9. user (2026-06-08T13:03:38.952Z)

Rally-detection methods ? research (2026-06-06)

Deep-research survey (23 sources, 110 claims ? 25 adversarially verified, 6 refuted, 12 final findings) on alternative ways to find rally start/end in racket-sport video, for our **local-first, permissive, single-GPU** pipeline. Question: is Gemini-teacher ? local-distillation the best, or are there competitive/ complementary methods?

Verdict
Teacher?student distillation is **sound but NOT uniquely best**. The evidence points to a **complementary multi-cue LOCAL stack**. VLM/MLLM teachers are validated specifically as **training-time priors**, not runtime localizers ? which is exactly our offline-Gemini framing. The single highest-leverage near-term lever is to **add a fully-local AUDIO hit-detection cue and fuse it with the shuttle trajectory**; pose/court serve-gating is the strong #2.

Three fully-local, permissive, competitive method families
1. **Audio hit detection (shuttle "thwack") ? top cheap lever.** Energy-peak + small-CNN gives sub-ms impact timing (peak precision ~0.98, recall ~0.95+); classical MFCC+GMM is CPU-light. ? Only as good as its post-processing: raw peaks ~41?50 F-score ? **likelihood-ratio confidence raises audio-only to ~68?77**; fully-automatic on-court systems degrade (~85%, and one tennis system hit 46% precision). Mostly validated on tennis/table-tennis (louder balls) ? shuttle SNR transfer is an open risk.
2. **Pose + court-geometry state machine** (= our "gate B" idea, validated). Serve-ready stance (court+pose features, BiLSTM/CNN over a 12×26 hand-engineered matrix) ? rally **START**; shuttle-flight gradient (?80% consecutive no-motion frames) ? rally **END**. Cheap (engineered features, no big model).
3. **Compact HitNet GRU** (MonoTrack, ~16K params, 2-layer GRU over 12-frame window of court corners + both players' poses + shuttle xy) ? consumes exactly our available inputs; trivially runnable. Detects *strokes* (a segmentation primitive), not whole rallies.

Key supporting evidence
- **Audio+visual fusion beats either alone:** tennis HMM rally **recall 83% (fused) vs 54% visual / 63% audio** at ~unchanged precision; soccer late-fusion +4?7% mAP. This is the core case for adding audio.
- **Distillation is legit:** MLLM4WTAL uses the MLLM as a training-time prior only; SSL+KD (COMEDIAN) is top-tier action-spotting. Our teacher?student is a valid backbone.
- **Our windowing is "coarse action spotting" (5?60 s tolerance), not frame-exact PES** ? don't over-engineer for frame precision.

? Caveats (honest)
- **Domain transfer:** nearly all high numbers come from **BWF/tennis BROADCAST** footage (replays, score graphics, director cuts) or curated lab audio. Our target is single-fixed-camera amateur video ? broadcast CNNs (97?99%) and scoreboard-OCR are inapplicable/degrade.
- **Metric mismatch:** headline numbers are **per-frame / per-component accuracy, NOT IoU-0.5 boundary F1**. The "~95% beats your 0.44" comparison was **refuted (0-3)** ? not comparable.
- **License gap (hard product req):** ShuttleSet, MonoTrack/HitNet weights, and WASB

weights commercial
  terms were NOT confirmed ? must verify before commercial use. (WASB code = MIT; weights = academic-trained.)
- Scoreboard/score-overlay OCR: usable only when a stable on-screen score exists ? mostly N/A for amateur clips.

Datasets
- **ShuttleSet** ? 104 sets / 3,685 rallies / 36,492 strokes / 44 BWF matches (2018?21), 18 shot classes +
  hitting/player locations. Stroke-level structure for train/eval, but broadcast-domain + license TBD.

Ranked recommendation (local-first, permissive, single-GPU)
Gemini-teacher ? local-distillation is a valid **backbone to AUGMENT, not rely on alone**. Cheapest local adds,
in order: **(1) audio hit detection fused with trajectory** (attacks the WASB recall bottleneck with inputs we
already have); **(2) pose/court serve-ready START + flight-gradient END gating**; **(3) a compact HitNet-style
GRU** on pose+trajectory+court. All MIT/permissive-compatible and CPU/GPU-light. Keep Gemini offline-only.

Concrete cheap next experiments (with what we already have)
- **E1 ? audio recall probe:** extract audio ? energy-peak detection (high-pass + EMA threshold) ? candidate
  impact timestamps ? overlay hit density on existing WASB windows ? does audio recover rallies WASB misses?
  (zero new training; directly tests the recall bottleneck.)
- **E2 ? audio FP suppression:** add likelihood-ratio confidence (raw peaks ~41?50 F ? ~68?77 F).
- **E3 ? fuse into the distillation student:** add audio-hit-rate / time-since-last-hit / applause-energy as
  features to the logistic-regression student ? re-eval at IoU 0.5.
- **E4 ? pose-free serve gate:** use the existing person detector for a coarse "both players settled in court"
  gate at window boundaries (proxy for the pose state machine, no pose model needed).

Open questions
1. ShuttleSet / MonoTrack-HitNet / WASB-weights **commercial license** terms (hard requirement).
2. Does shuttle-"thwack" audio detection hold on **real amateur single-camera** badminton (weaker SNR, gym
   reverb), or degrade below the tennis ~73?77 F audio-only ceiling?
3. On OUR footage at IoU 0.5, do audio/pose cues actually fix the **WASB recall** bottleneck ? or does WASB
   miss the underlying trajectory regardless of how windows are derived?
4. Most label-efficient path given n=2: more Gemini labels vs audio/pose weak-supervision vs a small
   hand-labeled golden set ? biggest IoU-0.5 lift per labeling hour?

Sources (primary)
Springer 978-3-031-69073-0_15 (+ SN CompSci 6(5):447) · MonoTrack arXiv:2204.01899 · arXiv:2409.11760 (audio
peaks) · Kijak tennis AV-HMM (irisa) · ShuttleSet arXiv:2306.04948 · broadcast-CNN arXiv:2308.12199 /
2307.16000 · MLLM4WTAL arXiv:2411.08466 · action-spotting survey arXiv:2505.03991 · WASB-SBDT (github nttcom)
· CoachAI license (github wywyWang).

10. user (2026-06-08T13:03:39.134Z)

CLAUDE.md ? project memory / agent handoff

Local, AI-powered **badminton (? racket-sports) highlight indexer + rally detector**:

```
FastAPI + SQLite + ffmpeg + GPU CV/AI, with paid **Gemini** as the cloud AI. Sibling
repos: `sports-data-collector`, a VLC-based `rally-annotator`.

## Read these first
- **`docs/README.md`** ? doc index. **`docs/CODE_MAP.md`** ? code navigation + data
  contracts (the cold-start map; read before touching code).
- **Strategy (post-scaffolding):** `docs/COMMERCIALIZATION.md`, `docs/PMF_MARKET_RESEARCH.md`,
  `docs/PLATFORM_ARCHITECTURE.md`, `docs/DEFERRED_HARDENING_PLAN.md`. Also `docs/ROADMAP.md`
  (offline-segmenter narrative) and `docs/BACKLOG.md`.
- **History:** `docs/archives/` ? ADRs (`decisions/DECISIONS.md`), research, and
  **`checkpoints/`** (latest: `2026-06-strategy-foundation/`). Archive policy: only move
  **terminal-state** (DONE/REJECTED) work; live docs stay at `docs/` top level.

## Current state (June 2026)
- **Scaffolding complete**; the **post-scaffolding strategy foundation** is laid and merged
  (#52/#53/#55/#57/#58). **No platform/owned-model implementation has started.**
- **Next slice = async job model (`DEFERRED_HARDENING_PLAN.md` #16), single-tenant.**
  **Do not start implementation without explicit owner approval.**

## Architecture in one breath
- **Pluggable registries** (ABC + `Registry` singleton): segmenters / providers / validators
  / sports / annotations ? see `backend/pipeline/segmenters/base.py`. *Future:*
  `StorageBackend` + `ComputeBackend` registries (PLATFORM §3).
- `video_id` = MD5 of the file ? `backend/utils/hashing.py::compute_video_id` (one helper).
- Typed config = `backend/config/` (Pydantic). DB schema evolves via the `PRAGMA
  user_version` migration ladder in `backend/infrastructure/database.py`.

## Committed guardrails (non-negotiable)
- **Paid LLMs (Gemini/OpenAI/Anthropic) = inference / serving / fallback ONLY ? never a
  training data source** (terms/copyright). Training teachers/bases = **open Apache-2.0 only**.
- **Owned-model base:** Apache-2.0, video-capable (Qwen-VL / Gemma 4); **reject Gemma 3**
  (viral license). **Exclude minors** from the training pool; per-user training data needs a
  separate explicit opt-in.
- **v1 = rally + highlight + history + correction loop**; **gait/biometrics strictly future**
  (GDPR Art. 9).
- **Generic data schema**; `coach ? athlete` is an **optional overlay**, not baked in.

## Workflow conventions
- **Branch off `master`; ONE PR per work item; NO PR stacking.** Don't open a PR unless asked.
- Tests: `python run_all_tests.py` (offline, fully mocked). Python 3.8+; `ffmpeg` required.
- Never push to the default branch directly; descriptive commit messages.
- ?? This dev container lacks GPU / torch / cv2 / numpy ? the **full pipeline can't run here**.
  Verify pure-logic paths and hand machine-side checks (real video, GPU/WSL suite) to the owner.
```

11. assistant (2026-06-08T13:03:40.940Z)

Perfect. I now have all the documentation. Let me synthesize this into a structured grounding brief for the owned-model training study.

12. user (2026-06-08T13:04:25.172Z)

Structured output provided successfully

13. assistant (2026-06-08T13:04:39.415Z)

Done. I've extracted and synthesized the grounding brief from the documentation. Here's what I found:

Summary

OWNED-MODEL INTENT (confirmed & clarified):
The project commits to building an Apache-2.0-clean, on-device-capable video-QA model for rally temporal segmentation. It is *not* about owning a VLM ? Qwen/Gemma are free to download. The moat is the **consented badminton-specific video?QA dataset + the eval harness**. The model plugs into the existing provider registry with Gemini as an always-ready fallback, evolving from Option A (modular, locally-fused cues on frozen detectors, 2?4 weeks) ? Option B (end-to-end fine-tuned multimodal model, 12?24+ months post-scaffold). Scaffolding (P0?P4: async jobs, multi-tenant, storage/compute abstraction, rich metadata) is the prerequisite; owned-model focus (P5) comes after.

HARD GUARDRAILS (non-negotiable):
1. **Training base = Apache-2.0 ONLY.** Approved: Qwen2.5-VL-7B/32B, Qwen3-VL-4B/8B, Gemma 4 multimodal. REJECTED: Gemma 3 (viral Google PUP), Llama Vision (700M-MAU cap + naming), gpt-oss (text-only).
2. **Training data = open-model outputs ONLY.** Paid APIs (Gemini, OpenAI, Anthropic) are inference/evaluation/teacher-reference only ? NEVER harvested as training data (competing-model breach).
3. **Exclude minors; separate opt-in.** Training consent is distinct from processing consent. Per-user adapters trained only on that user's own video.
4. **Generic schema.** Coach?athlete is optional overlay, not baked in.

DATA ASSETS REUSABLE NOW:
- 96 (Adarsh+Avi) + 32 (Kushagra) human-labeled rally [start,end] CSVs + ending-reason labels
- WASB shuttle-trajectory CSVs (MIT-licensed, frozen, permissive)
- Gemini silver labels (eval & teacher-reference only, not training)
- ~26 GB licensed/owned GoPro footage (1080p, 60fps, 22/22 matches re-acquired)
- Roora vendor (Bengaluru) initial 3-sport pilot underway

ROADMAP FRAMING (heuristics ? Option A ? Option B):
- **NOW (Option A, 2?4 weeks):** Modular, fully-local cues (WASB + audio hit-detection per ADR-007). Each swappable, testable. No owned-model training yet.
- **6?12 months (still Option A + scaffold P0?P4):** Async jobs + multi-tenant foundations. Incremental re-training as golden-set grows via Roora. Data-leakage guardrails (train/eval split by match).
- **12?24+ months (Option B):** Collapse modular cues into single fine-tuned end-to-end owned model. Requires healthy corpus + nanochat-style training harness + measured improvement bar.

STUDY SCOPE (immediate blockers on training-readiness):
1. Owned-model base selection (Qwen vs Gemma 4) + per-checkpoint commercial license verification.
2. Fine-tuning backend (self-managed ms-swift on rented GPU vs. managed Together/Fireworks vs. Predibase).
3. Golden QA eval harness design (nanochat-style pass/fail gate ? this is the crux).
4. Training data schema (rally CSV ? clip?QA pairs without paid outputs; minors exclusion).
5. Data-leakage guardrails + train/eval split enforcement.
6. Training orchestration (how it slots into async job model P0, ComputeBackend, BYO-compute).

The brief is now ready for a training-study design session.